Daffodil International University
Department of Software Engineering

System Design Document

Unit 7: Disputes, Trust & Safety, Staff Tools

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 18 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Depends on	Unit 1 (Identity & Access), Unit 2 (Order, Payment & Escrow), Units 3 to 6 (ports and events)
Database schema	`docs/design/sql/07_disputes_trust_staff.sql`, tested on PostgreSQL (87 checks and concurrency tests, Appendix B)

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Requirements covered . . . 3

2 Unit Overview **3**
2.1 Responsibilities . . . 3
2.2 Components . . . 4
2.3 The staff console . . . 4

3 Interfaces **5**
3.1 HTTP API . . . 5
3.2 Ports provided . . . 6
3.3 Events . . . 6
3.4 Ports required . . . 6

4 Data Design **7**
4.1 Tables and rules the database enforces . . . 7

5 Disputes **8**
5.1 Life cycle . . . 8
5.2 Opening . . . 9
5.3 The seller's response and the parties' statements . . . 10
5.4 The case file . . . 10
5.5 Decision and execution . . . 11
5.6 Four-eyes approval . . . 11
5.7 Withdrawal . . . 11
5.8 Review of a decision (FR-DSP-12, deferred) . . . 12

6 Trust & Safety **12**
6.1 Reports . . . 12
6.2 Flagged activity . . . 12
6.3 Sanctions . . . 13
6.4 Seller strikes and the automatic suspension . . . 13
6.5 A seller who recovers a sold game account . . . 13

7 Ratings **14**

8 Parameters and Feature Switches **14**

9 Privacy Functions **15**

10 Scheduled Jobs and Event Consumers **15**

11 Performance and Scaling **15**

12 Security **16**

13 Parameters **16**

14 Testing **17**

15 Notes for Integration **17**

A Traceability **18**

B Database Schema and Tests **19**

B.1 Test results 19

B.2 Schema 20

1 Introduction

1.1 Purpose

This document is the seventh and last backend unit of TrustBazaar's System Design Document. It covers what happens when something goes wrong, and the tools staff use to handle it:

- **disputes**: opening while the money is still held, the seller's response, the case file, the operator's decision and its execution through Unit 2's escrow;
- **trust & safety**: reports, flagged activity, sanctions, seller strikes and the automatic selling suspension, and the response to a seller who recovers a sold game account;
- **ratings**: ratings and reviews after an order ends, and the trust summary on profiles;
- **staff tools**: the operator queues and the fixed list of staff actions, versioned parameters and feature switches with their validation rules, and two privacy functions (data export and lawful disclosure).

Ratings were not assigned to any unit in the original plan. They are designed here because they are a trust signal fed by the same order outcomes that disputes and strikes use.

The unit follows DC-1 to DC-14. Many earlier units already rely on it: they call `DisputeCommands`, `ReportCommands`, `SanctionPort` and `ConfigPort`, and their operator actions appear in its queues. Unit 7 is therefore mostly coordination: where a fact belongs to another unit (the dispute window, the escrow, the chat thread), Unit 7 asks that unit and does not keep a copy.

1.2 Scope

In scope: disputes and their history, statements and evidence; reports; signals for the flagged-activity queue; sanctions, strikes and restrictions; ratings, replies and removals; the staff console, the queues and the action list; parameter versions and switches; data exports and disclosures.

Out of scope, owned by other units: escrow movements and the fee split (Unit 2); KYC review, the NID blacklist itself, account suspension, four-eyes approvals and the audit log (Unit 1); listing review, bid voiding and auction cancellation (Unit 3); return shipments (Unit 4); game-account evidence (Unit 5); chat threads and notifications (Unit 6); the console's screens (Unit 8).

1.3 Requirements covered

- **Functional:** FR-DSP-01 to FR-DSP-11 (FR-DSP-12 deferred, Section 5.8); FR-TNS-01 to FR-TNS-09; FR-REV-01 to FR-REV-07; FR-OPS-01, FR-OPS-02, FR-OPS-08 and FR-OPS-09, completing Unit 1's FR-OPS-03 to FR-OPS-07; FR-GAM-18 (sanction and export); FR-PRV-03, FR-PRV-06.
- **Security:** SEC-LOG-01 (case-file views); SEC-ACC-01 to SEC-ACC-05 (staff access); SEC-IRM-04 (emergency controls); SEC-ABU-01 (reports).
- **Other:** NFR-USE (queue age, decision preview); DR (dispute and sanction records).

2 Unit Overview

2.1 Responsibilities

1. Freeze the money the moment a dispute opens, decide it on a complete case file, and execute the decision exactly once.
2. Turn reports and signals from every unit into a small number of queues, and record every sanction with its reason and its end.

3. Suspend selling automatically after CP-47 seller strikes, exactly once, however the strikes arrive.
4. Show honest ratings: only from finished orders, one per direction, fixed after the edit period.
5. Let administrators change parameters only within the SRS rules, as new versions that never alter running orders.

2.2 Components

Component	Responsibility
DisputeService	Opening (with the window check and the freeze), responses, evidence, withdrawal, decisions, outcome (d) (Section 5).
CaseFileBuilder	Assembles the case file from Units 2 to 6 for the assigned operator; every view audited (Section 5.4).
ReportService, SignalIngest	Reports from users; signals from other units' events (Section 6).
SanctionService, StrikeService	Sanctions and their enforcement; strikes and the automatic suspension; implements `SanctionPort`.
RatingService	Eligibility from order outcomes; ratings, replies, removals; the profile summary (Section 7).
StaffConsole	Queue and action endpoints; claims with the conflict check (Section 2.3).
ConfigService	Versioned parameters and switches; validation; implements `ConfigPort` (Section 8).
PrivacyService	Data exports and disclosures (Section 9).
Unit7Jobs	Seller-response deadline, signal aggregation, export expiry (Section 10).

2.3 The staff console

FR-OPS-01 allows operators exactly eight queues, and FR-OPS-02 a closed list of actions. The console is one module that shows each queue and offers only the listed actions; the data and the action itself stay with the unit that owns them.

Queue (FR-OPS-01)	Owner	Actions offered (FR-OPS-02)
KYC submissions	Unit 1	Approve, reject, propose revocation (four-eyes).
Ownership proofs	Unit 5	Approve, reject (with the live-profile check).
Listings in review	Unit 3	Approve, return, remove.
Reports	Unit 7	Dismiss, or action: sanction, remove listing or review, void bid, cancel auction.
Flagged activity	Unit 7 (signals from Units 3–6)	Dismiss, sanction, propose blacklist; delivery issues: unlock a handover code, resolve (Unit 4).
Disputes	Unit 7	Ask for information, decide, export the evidence pack.
Bank payout-account checks	Unit 2	Approve, reject.
Payment discrepancies	Unit 2	Resolve.

Unit 4's delivery issues and Unit 6's chat flags appear as signals in *flagged activity*, not as queues of their own, so the list stays at the eight queues the SRS allows. Two actions designed in earlier units are not in FR-OPS-02's list: unlocking a handover code and resolving a delivery issue (Unit 4). The same applies to maintaining reference data: screening and detection terms by operators, and meetup points, areas and the games catalogue by administrators. An SRS clarification adds them (I-50).

Every action requires a reason, names its object and is audited by Unit 1's `AuditService` in the same transaction (FR-OPS-07). A staff member claims a case before acting; the claim runs Unit 1's

conflict check (FR-OPS-06), so an operator cannot take a dispute or report involving an account that shares their number, e-mail or NID, or one they have declared a conflict with. There is no generic create, edit or delete of users, listings, orders or payments anywhere.

3 Interfaces

3.1 HTTP API

All paths are under `/api/v1`; policies and rate limits are those of Unit 1.

Method	Path	Policy	Purpose (requirement)
Disputes (buyer and seller)			
`POST`	`/orders/{id}/disputes`	Buyer	Open: type, description (FR-DSP-01)
`GET`	`/disputes/{id}`	Buyer or seller	Status, deadlines, statements, own evidence, outcome
`POST`	`/disputes/{id}/statements`	Buyer or seller	Add a statement; the seller's first one is the response (FR-DSP-04)
`POST`	`/disputes/{id}/evidence`	Buyer or seller	Upload evidence, up to CP-33 per dispute
`POST`	`/disputes/{id}/withdraw`	Buyer	Withdraw before the decision (FR-DSP-10)
`POST`	`/disputes/{id}/return-received`	Seller	The returned item has arrived (outcome (d))
Reports and ratings			
`POST`	`/reports`	SignedIn	Report a listing, user or message (FR-TNS-01)
`POST`	`/orders/{id}/rating`	Buyer or seller	Rate the other party (FR-REV-01)
`PATCH`	`/ratings/{id}`	Rater	Edit within the edit period (FR-REV-03)
`POST`	`/ratings/{id}/reply`	Rated user	One public reply (FR-REV-06)
`GET`	`/users/{id}/trust`	Public	Average, count, completed transactions, badges (FR-REV-04, 05)
Privacy			
`POST`	`/me/data-export`	SignedIn, StepUp	Request a copy of my data (FR-PRV-03)
Staff (operators unless stated)			
`GET`	`/staff/queues/{queue}`	Staff(operator)	One of the eight queues, oldest first, with age
`POST`	`/staff/cases/{type}/{id}/claim`	Staff(operator)	Claim, with the conflict check (FR-OPS-06)
`GET`	`/staff/disputes/{id}/case-file`	Assigned operator	The case file; each view audited (FR-DSP-05)
`POST`	`/staff/disputes/{id}/request-info`	Assigned operator	Ask the seller for more (FR-DSP-09)
`POST`	`/staff/disputes/{id}/decision`	Assigned operator	Decide; proposes four-eyes approval when CP-30 applies (FR-DSP-06)
`POST`	`/staff/reports/{id}/resolution`	Assigned operator	Dismiss or action, with a reason
`POST`	`/staff/signals/{id}/review`	Staff(operator)	Dismiss or action
`POST`, `DELETE`	`/staff/users/{id}/sanctions`	Staff(operator)	Apply or lift a sanction (FR-TNS-03)
`POST`	`/staff/ratings/{id}/removal`	Staff(operator)	Remove a review breaking the policy (FR-REV-07)
`GET`	`/staff/disputes/{id}/evidence-pack`	Assigned operator	Export for law enforcement (FR-GAM-18)

Method	Path	Policy	Purpose (requirement)
`POST`	`/staff/disclosures`	Staff(operator)	Record a lawful request; proposes four-eyes (FR-PRV-06)
`GET`, `POST`	`/admin/config`	Staff(admin)	Current version; publish a change (FR-OPS-03, 08, 09)

3.2 Ports provided

```
interface DisputeCommands {                          // used by Unit 5, in its own transaction (DC-13)
  open(o: { orderId: string; type: DisputeType; description: string; by: 'buyer' | 'system' },
       tx: EntityManager): Promise<string>;          // credentials_invalid report, re-key overdue
}
interface ReportCommands {                           // used by Unit 6 (FR-CHT-06)
  report(r: { reporterId: string; target: ReportTarget; category: ReportCategory; text?: string }): Promise<
     string>;
}
interface SanctionPort {                             // used by Unit 1 (assertCanTransact) and Unit 2 (payouts)
  restrictions(userId: string): Promise<{ canSell: boolean; canBuyBid: boolean; fraudHold: boolean }>;
}
interface ConfigPort {                               // used by every unit (DC-9)
  current(): Promise<{ version: number; get<T>(code: string): T }>;
  atVersion(version: number): Promise<{ get<T>(code: string): T }>;   // a running order's own values
}
interface TrustQuery {                               // used by Units 3 and 8
  profileSummary(userId: string): Promise<{ averageRating: number | null; ratingCount: number;
                                            completedTransactions: number }>;
}
```

3.3 Events

Event	Use
Published (outbox, DC-6)	
`disputes.dispute.opened`, `.withdrawn`, `.decided`	Notifications to both parties (FR-DSP-03, FR-DSP-07); Unit 6 holds or releases the chat thread.
`trust.sanction.applied`, `.lifted`	Notifications with the reason (FR-TNS-03); Unit 3 hides or shows listings.
`trust.report.resolved`	Notification to the reporter: reviewed, and whether action was taken (FR-TNS-09).
`staff.config.published`	Every unit refreshes its cached parameters.
Consumed (idempotent)	
`orders.escrow.released`, `.refunded`, `.partially_refunded`	Rating eligibility (FR-REV-01); a refund at a missed fulfilment deadline is a seller strike (FR-TNS-04).
`orders.order.cancelled_by_seller`	Seller strike (FR-PAY-09, FR-TNS-04).
`delivery.return.delivered`	Outcome (d) completed.
`auctions.flag.raised`, `listings.listing.relisting_flagged` (new, I-45), `chat.message.flagged`, `delivery.issue.opened` (new, I-46)	Signals in the flagged-activity queue (Section 6.2).
`identity.account.closed`	Pending exports cancelled; ratings by the user shown as "Closed account".

3.4 Ports required

Port	Provider and use
`IdentityCommands`, `ApprovalService`, `AuditService`, `StaffQuery`, `FileVault` (Unit 1)	Account suspension and reinstatement; four-eyes; audit; conflict check; encrypted evidence files.
`OrdersPort`, `EscrowQuery`, `EscrowCommands` (Unit 2)	Order facts and lock (DC-14); `disputeAllowed`; `freezeForDispute`, `applyDisputeOutcome`, `withdrawDispute`; outcome preview from `payments.split`.
`ListingsCommands` (Unit 3)	Remove a listing for a sanction.
`DeliveryQueries`, `DeliveryCommands` (Unit 4)	`notReceivedAllowed`; the fulfilment record; `quoteReturn`, `bookReturn`.
`GameAccountQueries` (Unit 5)	`credentialsInvalidAllowed`; `evidence`.
`ChatCommands`, `Notifier` (Unit 6)	Holds and the staff read; notices to parties and reporters.

4 Data Design

The schema is in `docs/design/sql/07_disputes_trust_staff.sql` (Appendix B), loaded after Units 1 to 6. The unit owns three schemas: `disputes`, `trust` and `staff`. Their foreign keys point only to users, staff accounts and approvals (Unit 1) and to orders (Unit 2), as DC-5 allows. Listings, threads and messages are referred to by identifier only. No function reads another unit's tables (DC-1). Facts that belong to other units, such as "the dispute window is open" or "the order's escrow is `held`", arrive as arguments that the service obtained from the owning unit's port in the same transaction.

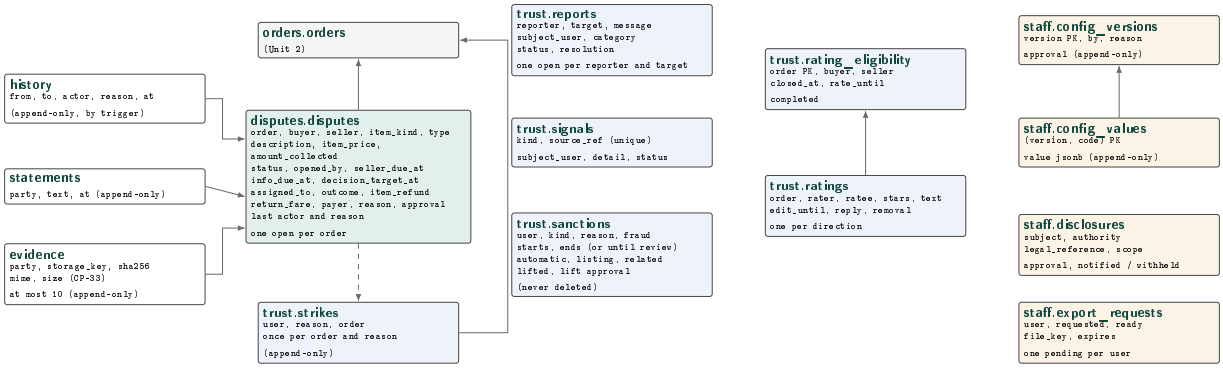


Figure 1: The schemas of Unit 7. Users and staff accounts (Unit 1) are referenced by most tables; disputes, sanctions, parameter versions and disclosures also reference Unit 1's approvals, and rating eligibility references the order. Those arrows are omitted. A strike references its order only; the dashed arrow shows that a lost dispute creates one.

4.1 Tables and rules the database enforces

Table	Rules enforced by the database
`disputes`	Buyer and seller differ. The type fits the item: game-account types only for game accounts, physical types only for physical items (FR-DSP-01). A description of at least 20 characters. One open dispute per order (FR-DSP-11). A dispute starts as `open`, and a type that was withdrawn on the order cannot be opened again (FR-DSP-10). What the dispute is about (order, parties, type, description, amounts, opening) is fixed. Only the transitions of Figure 2. Every status change names its actor and reason. A decision needs an outcome, the operator, the time and a reason of at least 20 characters, and is final. A partial refund is above zero and below the item price (FR-PAY-11); a refund after return names the fare and who pays it. A decision on BDT 20,000 or more needs an executed four-eyes approval (CP-30).
`history`	One row for every status change, written by a trigger, so it cannot be skipped (FR-DSP-09). Append-only.
`statements`, `evidence`	Append-only. Evidence: the allowed file types, at most 10 MB each, at most 10 per dispute (CP-33), and only while the dispute is being considered.
`reports`	A user cannot report themselves. A reported message names the message. One open report per reporter and target. A resolution records the operator, the time and a reason.
`signals`	One row per source event (kind and source reference unique), so an event delivered twice is stored once. A review records the operator.
`sanctions`	A reason of at least 10 characters. A warning and a listing removal have no duration; a listing removal names the listing. A sanction applied by an operator names the operator. A sanction cannot be rewritten: its end changes only by lifting it, with a reason; lifting an account suspension needs four-eyes approval (CP-30). At most one live automatic suspension per user. Never deleted.
`strikes`	One strike per order and reason. Append-only.
`rating_eligibility`, `ratings`	A rating exists only for an order in `rating_eligibility`, that is, one whose escrow reached a terminal state (FR-REV-02). Only the buyer and the seller rate each other, once in each direction, within the CP-39 window. 1–5 stars. Stars and text change only within the edit period and are then fixed (FR-REV-03). One reply, only to a written review, never changed (FR-REV-06). A removal records the operator and a reason, and is final (FR-REV-07).
`config_versions`, `config_values`	Append-only. A code is a CP or SW identifier. Version 1 is the seed; every later version names its administrator and a reason.
`disclosures`	Nothing is disclosed without an approval. The user is either told, or the reason not to tell is recorded (FR-PRV-06).
`export_requests`	One export in preparation per user. A ready export has its file and its expiry.

The application role cannot update or delete the history, statements, evidence, strikes or parameter versions, and cannot delete sanctions (tested).

5 Disputes

5.1 Life cycle

The six statuses are the SRS's (FR-DSP-09). While the dispute is `open`, the seller has CP-20 (48 hours) to respond. The review starts when the seller responds, or when CP-20 passes without a response, in which case the operator decides on the evidence available (FR-DSP-04). The review start sets the decision target, CP-21 after it (FR-DSP-08). The queue shows the age of each dispute

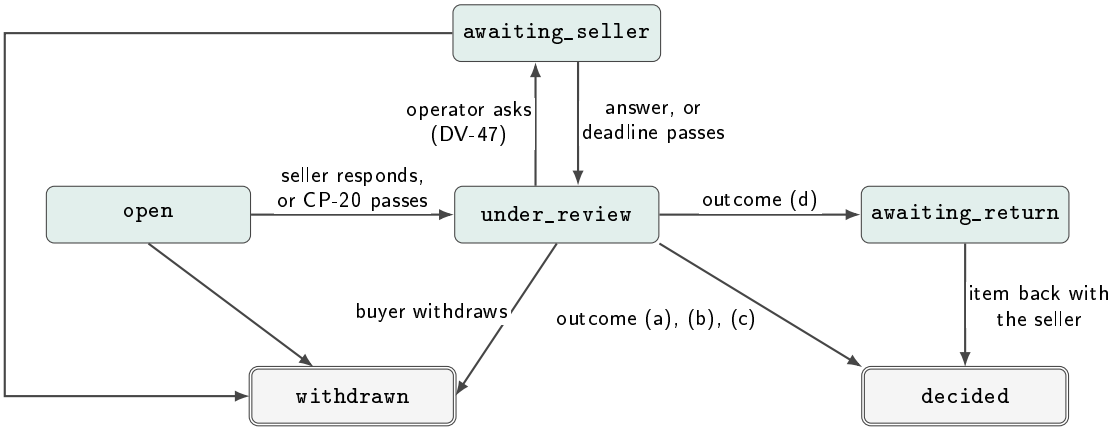


Figure 2: Dispute statuses (FR-DSP-09). The buyer can withdraw from `open`, `under_review` and `awaiting_seller`. Every arrow writes a history row with its actor and reason.

and puts overdue ones first (DV-52). During the review the operator may ask the seller for more; the dispute is then `awaiting_seller` until the seller answers or the operator's deadline passes (DV-47, 24 hours), and returns to review either way. The decision target is not reset by such a request: the CP-21 clock measures the operator, not the seller.

5.2 Opening

The buyer opens a dispute from the order view with a type, a description and evidence (FR-DSP-01). Two kinds of dispute are opened by the system rather than the buyer: Unit 5 opens a `credentials_invalid` dispute when the buyer reports that the credentials do not work (FR-GAM-14), and when the seller does not answer a re-key in time (DV-37). Both go through `DisputeCommands.open`. Who may open which type, and when, is decided by the unit that owns the deadline:

Type	Window (SRS)	Answered by
`not_as_described`, `damaged`, `other (physical)`	CP-18 after the delivery event	Unit 2 `EscrowQuery.disputeAllowed`: `held` and inside the window.
`not_received`	CP-17 after pickup with no delivery (FR-DLV-11)	Unit 4 `notReceivedAllowed`, and Unit 2 for `held`.
`credentials_invalid`	The takeover window, until the buyer confirms the new recovery e-mail (FR-GAM-14)	Unit 5 `credentialsInvalidAllowed`, and Unit 2 for `held`. Opened by Unit 5 itself.
`seller_recovery`, `other (game)`	The protection period CP-24 (FR-GAM-16)	Unit 2 `disputeAllowed`.

After the payment has been released or refunded, no dispute can be opened; later complaints are reports (FR-DSP-11). A seller who recovers an account after release is handled through a report and the sanction of Section 6.5.

Opening is one transaction, with the locks in the order of DC-14, the dispute row taking the place of the delivery row:

1. lock the order through `OrdersPort.lockForFulfilment`, which also gives the parties, the item kind and the amounts;
2. ask the owning unit whether this type may be opened now (table above);
3. `disputes.open`, which inserts the dispute (refused if one is open, or if this type was withdrawn), writes the first history row and the `disputes.dispute.opened` event;

4. `EscrowCommands.freezeForDispute`: the escrow moves to `frozen_disputed`, which stops every automatic release (FR-DSP-02);
5. `ChatCommands.hold` on the order's thread, so it is kept and the assigned operator may read it (FR-CHT-10);
6. the evidence files, already uploaded to Unit 1's `FileVault` with the evidence purpose, are recorded.

If any step fails, nothing is kept: no dispute without a frozen escrow, and no frozen escrow without a dispute. Ten simultaneous openings on one order produced one dispute and nine refusals (Section 14). The event notifies the seller (FR-DSP-03) and the dispute appears in the queue.

5.3 The seller's response and the parties' statements

The seller's first statement is the response: it moves the dispute to review (FR-DSP-04). Either party may add statements and evidence until the decision, up to CP-33 (10) files per dispute in total. A response after CP-20 is refused, and the review starts on the evidence available. The CP-20 job checks every five minutes; a seller who answers one minute after the deadline is refused even if the job has not yet run, because the function compares the time itself.

5.4 The case file

The assigned operator sees one case file (FR-DSP-05). Each item comes from the unit that owns it, through its port; Unit 7 keeps no copies.

Item (FR-DSP-05)	Unit	Source
The order and the listing as they were at the time of sale	2	The order with its listing snapshot (Unit 2 stores the snapshot at checkout).
The escrow history	2	`EscrowQuery.getEscrow`, with its state changes.
The settlement and ledger entries	2	The order's ledger entries and the preview of each outcome from `payments.split`, so the operator sees the exact amounts before deciding.
The delivery or handover records	4	`fulfilmentRecord`: proposals, code entries (never the code), shipments, courier history, issues.
The chat thread for the listing	6	`chat.staff_read`, allowed only by the dispute's own hold; each read logged.
Both parties' evidence	7	Statements and evidence; files through `FileVault` signed links (CP-31).
For game accounts: the ownership proof, takeover confirmations and credential-access log	5	`GameAccountQueries.evidence`. Never the credentials.
Context (not in the SRS list)	7	Earlier disputes, strikes and sanctions of both parties.

Personal data in the case file is masked by default (SEC-ACC-05). Every view of the case file is written to the audit log (SEC-LOG-01). Only the operator assigned to the dispute can open it, and assignment passes Unit 1's conflict check (FR-OPS-06). In the race test, ten operators claimed one dispute at once; one got it and nine were refused.

5.5 Decision and execution

The operator chooses one of the four outcomes of FR-DSP-06 and writes a reason; the parties see the reason (FR-DSP-07). The service executes the decision in one transaction, locking the order, then the dispute, then the escrow:

Outcome	Execution
(a) Release	`disputes.decide`, then `applyDisputeOutcome(release)`: the seller is paid as on a normal release.
(b) Full refund	`applyDisputeOutcome(full)`. Unit 2 refunds the item price, and the delivery fee too if the courier never picked up.
(c) Partial refund	`applyDisputeOutcome(partial, itemRefund)`, with the item refund above zero and below the item price (FR-PAY-11).
(d) Refund after return	`DeliveryCommands.quoteReturn` and `bookReturn` with the payer the operator chose; `applyDisputeOutcome(after_return)` keeps the escrow frozen. The dispute is `awaiting_return`. When Unit 4 reports `delivery.return.delivered`, or the seller records receipt, `return_completed` closes it and Unit 2 makes the refund (I-44).

Any outcome other than release is a lost dispute for the seller, and records a strike (FR-TNS-04, Section 6.4) in the same transaction. A partial refund counts as lost, because the buyer's complaint was found at least partly justified. The `disputes.dispute.decided` event notifies both parties and releases the chat hold.

A decision cannot be changed. Neither the operator nor anyone else can change the outcome, the amounts or the reason afterwards; the database refuses it.

5.6 Four-eyes approval

CP-30 requires four-eyes approval for "dispute outcomes of BDT 20,000 or more". The design applies the threshold to the amount the buyer paid (`amount_collected`), because that is what any outcome can move. For such a dispute, the decision endpoint does not decide: it proposes the decision through Unit 1's `ApprovalService` (action `dispute.outcome_20000`, with the outcome, amounts and reason as the payload). A second operator who is not the proposer approves it, and the approval's handler, registered by Unit 7, runs the decision with the approval's identifier in the approval's transaction. Without an executed approval the database refuses the decision (tested). Until the approval, the dispute stays `under_review` and the buyer can still withdraw; if the buyer withdraws, the approval fails when it runs, because the dispute is no longer under review.

5.7 Withdrawal

The buyer may withdraw before the decision (FR-DSP-10). In one transaction: `disputes.withdraw`; `EscrowCommands.withdrawDispute`, which returns the escrow to `held` with automatic release at the later of the original time and 24 hours after the withdrawal; for game accounts, `GameAccountCommands.disputeWithdrawn`, which returns the handover to its state before the dispute; and the release of the chat hold. A withdrawn type cannot be opened again on that order, but another type can (tested).

Decision and withdrawal can arrive at the same moment. Both lock the dispute row, and whichever comes second finds the dispute closed and is refused. In the race test, 20 disputes were each decided and withdrawn at once: every dispute ended either decided or withdrawn, never both, never neither.

5.8 Review of a decision (FR-DSP-12, deferred)

FR-DSP-12 (a Could requirement) lets a party ask for one review of a decision by a different operator within three days. It is deferred. A decision is executed at once, so the escrow is already released or refunded when a review could be asked for, and a changed outcome would have to take money back from a party who may already have withdrawn it. Doing this properly needs either a delay before every execution, which slows every dispute, or a recovery process through Unit 2's `seller_receivable`, which only works in one direction. The schema keeps the decision final; adding the review later means a new `review_requested` status before execution, and does not change any other table. This is listed as an SRS clarification (I-50).

6 Trust & Safety

6.1 Reports

Any registered user can report a listing, a user or a chat message as fraudulent, counterfeit, prohibited, misrepresented or abusive (FR-TNS-01); chat reports arrive through `ReportCommands` (FR-CHT-06). The report names the user it is about, so the queue can group reports by user, and the conflict check can be made at claim. A reporter may have one open report per target; a second one adds nothing and is refused, which stops one person from pushing a listing up the queue. Reports are also rate-limited by CP-35 (20 a day). A report on a message or thread places a hold on the thread (Unit 6), so the assigned operator may read it.

The operator resolves a report as *dismissed* or *actioned*, with a reason. Actions are those of FR-OPS-02: a sanction, removing the listing or a review, voiding a bid, or cancelling an auction; each is carried out by the owning unit. The `trust.report.resolved` event tells the reporter that the report was reviewed and whether action was taken, without the details of any sanction (FR-TNS-09). The text is the same for every kind of action: "Thank you. We reviewed your report and took action" or "... and found no breach of our rules".

6.2 Flagged activity

The flagged-activity queue holds *signals*: things another unit noticed that an operator should look at. A signal is stored once per source event, however often the event is delivered.

Signal	From	Meaning
Shill bidding	Unit 3	The shill score of an auction (FR-AUC-18), including bidders linked to the seller (FR-TNS-08).
Relisting	Unit 3	A similar item relisted shortly after a cancelled or disputed sale (FR-TNS-02).
Linked accounts	Units 1, 2	Accounts sharing a device, a payout account or a payment instrument (FR-TNS-08), from a daily check through their query ports.
Chat	Unit 6	Off-platform or credential warnings, aggregated per user per day (DV-51): one signal, with the count, rather than one per message.
Delivery issue	Unit 4	A handover locked by wrong codes, or a courier problem. The operator unlocks or resolves it through Unit 4's actions.

An operator dismisses a signal or acts on it, with a note. Acting usually means a sanction, or proposing a blacklist entry through Unit 1.

6.3 Sanctions

An operator may apply the five sanctions of FR-TNS-03. Each has a reason, and an end date or "until review" (no end), and the user is told the reason. The restrictions are computed from the live sanctions by `trust.restrictions`, which `SanctionPort` exposes. Each unit applies them where it acts:

Sanction	Effect and where it is enforced
Warning	A notice only. Recorded, so it appears in the case files of later disputes.
Listing removal	Unit 3 removes the listing (`ListingsCommands`); a notice with the reason.
Selling suspended	No new listings, and the user's listings hidden (Unit 3, on `trust.sanction.applied`). Unit 1's `assertCanTransact` refuses selling actions using `SanctionPort`.
Buying suspended	No bids or purchases: Unit 1's `assertCanTransact`, called by Units 2 and 3.
Account suspended	All of the above, and Unit 1's `suspendAccount`, which ends the user's sessions. Lifting it needs four-eyes approval (CP-30), through `reinstateAccount`.
Fraud flag (any suspension)	Payouts to the user are held until an operator reviews them (FR-TNS-05): Unit 2's payout job asks `SanctionPort` for `fraudHold`.

Orders already paid continue to completion under the normal escrow rules (FR-TNS-05): a suspended seller still has to deliver, and a suspended buyer can still confirm receipt or open a dispute. A sanction cannot be edited. Shortening or ending it is a lift, with a reason; a different sanction is a new one. Expired sanctions stop counting by themselves, without a job (tested).

6.4 Seller strikes and the automatic suspension

A seller gets a strike for each of the three events of FR-TNS-04:

- a unilateral cancellation after payment (`orders.order.cancelled_by_seller`, FR-PAY-09);
- a missed fulfilment deadline (the refund event of Unit 2 with that reason, I-44);
- a lost dispute (Section 5.5).

A strike is stored once per order and reason, so a repeated event adds nothing. `trust.record_strike` counts the seller's strikes in the CP-47 window (90 days). At the CP-47 count (3), it applies a `selling_suspended` sanction marked automatic, "until review". It does so once: further strikes while that suspension is live add strikes but no second suspension. The function holds a lock for the seller while it counts. In the race test, ten strikes for one seller arrived at once; the result was ten strikes and exactly one suspension. An operator reviews the automatic suspension from the flagged-activity queue and lifts it or replaces it with a decided sanction.

Unit 3's bidder strikes for unpaid auction wins (CP-12) are a different thing: they restrict bidding, are kept by Unit 3, and do not count here.

6.5 A seller who recovers a sold game account

When an operator confirms that a seller recovered a sold account (FR-GAM-18), either by deciding a `seller_recovery` dispute for the buyer or by actioning a report after release, the console runs three steps:

1. an `account_suspended` sanction with the fraud flag, at once, which also holds the seller's payouts;
2. `IdentityCommands.proposeBlacklist` with the order and the reason, so that the seller's NID blind index is added to the blacklist when a second staff member approves (FR-TNS-06, CP-30);

3. the evidence pack: the ownership proof, the credential-access log and the takeover confirmations (Unit 5's `evidence`), and the dispute's history, statements and evidence. It is one PDF with a hash list of the files, created on request and audited, and served by a signed link.

7 Ratings

When an order's escrow reaches a terminal state (released, refunded or partially refunded), the Unit 2 event creates its row in `rating_eligibility`, with the CP-39 window (14 days). Only then can the buyer and the seller rate each other, once each, with 1–5 stars and optional text (FR-REV-01, FR-REV-02). An order that was never paid never reaches a terminal escrow state and so can never be rated. A rating can be edited for 48 hours (CP-39) and is then fixed (FR-REV-03). The rated user may post one reply to a written review (FR-REV-06, Could). An operator may remove a review that breaks the content policy, with a reason; it then no longer counts (FR-REV-07).

The profile summary shows the average of the ratings not removed, their number, and the number of completed transactions (FR-REV-04): orders released to the seller, where the user was either party. Refunded orders can be rated, but do not count as completed. The verified and Pro badges (FR-REV-05) come from Unit 1 and Unit 2 and are added by the API. Ratings are shown with the rater's name, or "Closed account".

8 Parameters and Feature Switches

Every configurable parameter of the SRS (CP-01 to CP-47) and the two emergency switches of SEC-IRM-04 (`SW-new_orders`, `SW-payouts`) live in one versioned set. Version 1 is seeded with the SRS defaults. An administrator changes the set by publishing a new version: the previous values with the changes applied (FR-OPS-03). Versions are never edited (FR-OPS-08).

- **Validation** (FR-OPS-09). `staff.config_violations` checks the complete candidate set against every rule of the SRS parameter table, including rules between parameters (CP-19 longer than CP-18; Pro commission not above standard; Pro limit not below standard; the CP-30 list may only grow). All broken rules are reported together. An unknown parameter is refused.
- **Four-eyes.** A change to a switch, to CP-36 (the game-account feature) or to CP-30 needs an executed approval (SEC-IRM-04, CP-30). The proposal goes through `ApprovalService`; the handler publishes the version with the approval.
- **Running items keep their values.** Units 2 and 3 store the version on each order, listing and auction at creation, and read their values through `ConfigPort.atVersion` (FR-OPS-08). New items use `current`.
- **Caching.** Each instance caches the current version, and refreshes on `staff.config.published` and at least every 60 seconds (DV-50). A new order may therefore be created on the previous version for up to a minute after a change. This is acceptable, because the order keeps whichever version it was created with. The emergency switches are the exception: they are read without the cache, so that stopping new orders takes effect at once.

Publishing takes a lock, so concurrent publications produce consecutive versions, each complete (tested with ten at once). The other administrator actions of FR-OPS-03 (staff accounts, MFA resets, the audit-log view) are Unit 1's.

9 Privacy Functions

Data export (FR-PRV-03, Should). A signed-in user, after step-up, requests a copy of their data. A job collects it through each unit's export query: profile and consents (Unit 1), orders and payments (Unit 2), listings and bids (Unit 3), addresses and shipments (Unit 4), game-account listings and handovers without any credential (Unit 5), own chat messages and notifications (Unit 6), and own disputes, reports, ratings and sanctions (Unit 7). The result is one JSON file in a ZIP, stored encrypted, offered through a signed link for 7 days (DV-49). One export may be in preparation at a time, and one may be requested every 7 days. KYC images are not included; the export says they can be requested from the operator.

Lawful disclosure (FR-PRV-06). An operator records a request from an authority with its legal reference and its scope. Nothing is disclosed until a second staff member approves it (four-eyes, recorded in Unit 1). The user is told, unless the law forbids it; then the reason is recorded instead. Both steps are audited.

10 Scheduled Jobs and Event Consumers

Job	Every	Action
Response deadlines	5 min	`seller_window_ended` for disputes past CP-20 or the information deadline: the review starts on the evidence available.
Decision targets	1 hour	Marks disputes past CP-21 as overdue in the queue and alerts the administrators once a day (FR-DSP-08).
Signal aggregation	daily	Chat flags into one signal per user per day (DV-51); the linked-account check (FR-TNS-08).
Data exports	5 min; daily	Build requested exports; delete expired files (DV-49).
Outbox dispatcher	continuous	Publishes `disputes.outbox` (DC-6).

Each job claims its rows with `SKIP LOCKED`, so several instances can run it. Each consumer is idempotent: rating eligibility is keyed by the order, strikes by order and reason, signals by their source event, and `return_completed` refuses a dispute that is no longer awaiting a return.

11 Performance and Scaling

- **Volume.** Disputes, reports and sanctions are few compared with orders and messages. Every queue is read through a partial index on open items in age order, so its cost depends on the open items, not on the history.
- **Hot paths.** `SanctionPort.restrictions` is called on every bid and checkout. It reads the user's live sanctions through a partial index; the result may be cached for 30 seconds per user and is cleared on `trust.sanction.applied`. `ConfigPort.current` is served from memory (DV-50).
- **The case file** calls five units. The calls are made in parallel, each a single indexed read, and the page shows each section as it arrives.

12 Security

Threat	Control
An operator decides a case of their own, or of a friend	Claim with Unit 1's conflict check (FR-OPS-06); only the assigned operator decides; four-eyes at CP-30; every action audited with its reason.
An operator moves a large amount to an accomplice	Four-eyes above BDT 20,000 in the database; the outcome preview shows the exact split; decisions cannot be changed afterwards.
An operator browses case files or chats	Only the assigned operator opens a case file; chat only through the dispute's hold; every view audited; personal data masked (SEC-ACC-05).
A buyer abuses disputes to delay payment	One open dispute per order; a withdrawn type cannot return; disputes only inside their window; withdrawal restarts release with at least 24 hours. Repeated losing disputes by a buyer are visible in later case files.
A seller delays the decision	CP-20 ends the response period; the review then starts without the seller.
A group reports a competitor	One open report per reporter and target; CP-35 report limit; reports never act by themselves.
An administrator weakens the rules	Every rule of the parameter table checked in the database; the CP-30 list can only grow; switches and CP-36 need four-eyes; every version kept with its author and reason.
A suspended user keeps selling	Restrictions checked in Unit 1's `assertCanTransact` on every protected action; listings hidden by Unit 3; sessions ended for an account suspension.
History rewritten	Dispute history, statements, evidence, strikes and parameter versions append-only; sanctions never deleted; the application role cannot change them.

13 Parameters

SRS parameters used: CP-17, CP-18 (windows, answered by Units 4 and 2); CP-20, CP-21 (response and decision target); CP-24 (protection period, Unit 2); CP-30 (four-eyes); CP-31 (signed links); CP-33 (evidence); CP-35 (report limit); CP-39 (ratings); CP-47 (strikes); and every parameter for validation.

Design values introduced (continuing DV-1 to DV-46):

ID	Value	Default	Reason
DV-47	Deadline for the seller's answer to an operator's question	24 hours	Half of CP-20; the CP-21 clock keeps running.
DV-48	Reports	one open per reporter and target; generic reporter feedback	No queue pushing; no sanction details (FR-TNS-09).
DV-49	Data export	link valid 7 days; one request per 7 days	The file holds personal data.
DV-50	Parameter cache	refresh on the event and at least every 60 s; switches not cached	Items keep their version anyway.
DV-51	Chat signals	one per user per day, with the count	One message per signal would flood the queue.
DV-52	Dispute queue order	overdue first, then oldest first	FR-DSP-08 shows the age.

14 Testing

Level	Cases
Database (done)	87 checks in `07_disputes_trust_staff_tests.sql`, each rejection required to fail on its intended rule: opening only inside the window, types that fit the item, one open dispute per order; permitted transitions only; the seller's response and its deadline; the operator's question; evidence limits; assignment; decisions with reasons, partial amounts, the return outcome and its completion; four-eyes above BDT 20,000; withdrawal and the no-reopening rule; history for every change; strikes, the automatic suspension once, and its effects; sanction rules, expiry, lifting with approval; reports and signals; ratings, edits, replies and removals, and the profile summary; parameter versions, every validation rule tried, switches with and without approval; disclosures and exports; grants. The Unit 1 to 6 scripts still pass with this schema loaded.
Concurrency (done)	Six races, three runs each on fresh databases, same results every time: 10 openings on one order (one dispute); 20 disputes decided and withdrawn at once (each one or the other, never both); 10 operators claiming one dispute (one assigned); 15 evidence uploads (10 stored); 10 strikes for one seller (one suspension); 10 parameter publications (versions 2 to 11, each complete).
Unit	The case-file builder with each unit's port failing; the approval handlers; the strike consumer with each event; the reporter's notice text.
Integration	Opening freezes the escrow and holds the chat; each outcome reaches the right escrow state in Unit 2, including (d) through a return shipment; withdrawal restores auto-release; a suspended seller is refused by `assertCanTransact` and their listings disappear; the fraud flag holds a payout; a published parameter reaches another instance within 60 seconds; a new order keeps the version it was created with.

15 Notes for Integration

I-43 Unit 1. `assertCanTransact` must call `SanctionPort.restrictions` (selling, or buying and bidding). Register the approval handlers `dispute.outcome_20000`, `account.reinstate`, `emergency.control` and `disclosure` for Unit 7. Provide the conflict check as a query used at claim. All units read parameters through `ConfigPort`.

I-44 Unit 2. The refund event at a missed fulfilment deadline must carry its reason, so it can count as a strike. Add `EscrowCommands.completeReturnRefund(orderId, disputeId, tx)` for outcome (d), called when the return is recorded. The payout job must ask `SanctionPort` for `fraudHold`. Escrow terminal events create rating eligibility.

I-45 Unit 3. Publish `listings.listing.relisting_flagged`. Hide and show a user's listings also on `trust.sanction.applied` and `.lifted` for selling and account suspensions. Provide `ListingsCommands.remove(listingId, reason, staffId)` for the listing-removal sanction.

I-46 Unit 4. Publish `delivery.issue.opened`, so delivery issues appear as signals in the flagged-activity queue rather than a queue of their own. `bookReturn` is called with the payer from the decision; `delivery.return.delivered` completes outcome (d).

I-47 Unit 5. `DisputeCommands.open` is implemented as designed, for the `credentials_invalid` report and the overdue re-key (`opened_by = system`). `evidence` feeds the case file and the FR-GAM-18 pack.

I-48 Unit 6. Holds are placed when a dispute opens or a report concerns a thread, and released on decision, withdrawal or resolution. `ReportCommands` is implemented here. Templates are needed for the dispute, sanction and report events.

I-49 Unit 8. The staff console: the eight queues with ages; claim; the case file with masked data;

the decision form with the outcome preview and the four-eyes notice; the parameter editor that shows every broken rule. For users: the dispute flow from the order view with deadlines as countdowns, ratings, reports and the data export.

I-50 SRS clarifications proposed:

- FR-OPS-02 and FR-OPS-03: add unlocking a handover code and resolving a delivery issue; maintaining screening and detection terms (operators); maintaining meetup points, areas and the games catalogue (administrators);
- FR-OPS-01: delivery issues and chat flags are part of *flagged activity*;
- CP-30: the BDT 20,000 threshold is measured on the amount the buyer paid;
- FR-TNS-04: a partial refund counts as a lost dispute;
- FR-DSP-12: deferred, because a decision is executed at once;
- add DV-47 to DV-52 to the SRS parameter table.

A Traceability

Requirement	Design element	Section
FR-DSP-01	`disputes.open`; window answered by the owning unit; `dispute_type_fits_item`	5.2
FR-DSP-02	`freezeForDispute` in the opening transaction	5.2
FR-DSP-03	`disputes.dispute.opened`; dispute queue	5.2
FR-DSP-04	`seller_respond`; `seller_window_ended` (CP-20)	5
FR-DSP-05	CaseFileBuilder from Units 2 to 7; audited views	5.4
FR-DSP-06	`disputes.decide`; `applyDisputeOutcome`; return booking for (d)	5.5
FR-DSP-07	`disputes.dispute.decided` with the reason	5.5
FR-DSP-08	Decision target (CP-21); age and overdue order in the queue (DV-52)	5
FR-DSP-09	Six statuses; guard trigger; `disputes.history` by trigger	5
FR-DSP-10	`disputes.withdraw`; `withdrawDispute`; no reopening of a type	5.5
FR-DSP-11	`disputes_one_open`; no dispute after release or refund	5.2
FR-DSP-12	Deferred (Could)	5.8
FR-TNS-01	`trust.reports`; `ReportCommands`	6
FR-TNS-02	Relisting signal from Unit 3	6.2
FR-TNS-03	`trust.sanctions`; reasons, ends, notices	6.3
FR-TNS-04	`trust.record_strike`; automatic suspension at CP-47	6.4
FR-TNS-05	`trust.restrictions`; `SanctionPort`; fraud hold on payouts	6.3
FR-TNS-06	Blacklist proposal through Unit 1	6.5
FR-TNS-07	Unit 2 (CP-27); the parameter is validated here	8
FR-TNS-08	Linked-account signal	6.2
FR-TNS-09	`trust.report.resolved`; generic text (DV-48)	6
FR-REV-01	`rating_eligibility` from escrow terminal events; CP-39 window	7
FR-REV-02	Foreign key to eligibility; `rating_once`	7
FR-REV-03	Edit period in `rating_guard`	7
FR-REV-04	`trust.profile_summary`; `TrustQuery`	7
FR-REV-05	Badges from Units 1 and 2	7
FR-REV-06	One reply, fixed	7
FR-REV-07	Removal with reason; excluded from the average	7
FR-OPS-01	Eight queues in the console	2.3
FR-OPS-02	Closed action list; reasons; no generic editing	2.3
FR-OPS-03	Parameter and switch changes by administrators	8
FR-OPS-05	Four-eyes for large outcomes, reinstatement, switches	5.6
FR-OPS-06	Conflict check at claim	2.3
FR-OPS-07	`AuditService` in each action's transaction	2.3
FR-OPS-08	`staff.config_versions`; `ConfigPort.atVersion`	8
FR-OPS-09	`staff.config_violations`	8
FR-GAM-18	Suspension with fraud flag; blacklist proposal; evidence pack	6.5
FR-PRV-03	`staff.export_requests`; export job (DV-49)	9

Requirement	Design element	Section
FR-PRV-06	`staff.disclosures` with approval and notice	9
SEC-LOG-01	Case-file views and staff actions audited	5.4
SEC-ACC-05	Masked personal data in the case file	5.4
SEC-IRM-04	Emergency switches with four-eyes	8

B Database Schema and Tests

B.1 Test results

Loaded after the Unit 1 to Unit 6 schemas in PostgreSQL 16, using PostgreSQL 15 features only. Every check passed.

```
PASS  FR-DSP-01 no dispute outside its window, or when the payment is not held [cannot be opened now]
PASS  FR-DSP-01 a game-account dispute type is not accepted for a physical item [dispute_type_fits_item]
PASS  FR-DSP-01 a physical dispute type is not accepted for a game account [dispute_type_fits_item]
PASS  a dispute needs a real description [disputes_description_check]
PASS  FR-DSP-01 a dispute opens as open, with the seller's CP-20 deadline, a history row and an event
PASS  FR-DSP-11 only one dispute open on an order at a time [already open]
PASS  FR-GAM-16 a seller_recovery dispute is accepted for a game account
PASS  a dispute cannot jump to decided [is not permitted]
PASS  what a dispute is about is fixed [is fixed]
PASS  FR-DSP-04 only the seller responds [only the seller]
PASS  FR-DSP-04 the seller's response starts the review, with the CP-21 decision target
PASS  CP-33 at most 10 evidence files per dispute [at most 10 evidence files]
PASS  evidence cannot be altered [append-only]
PASS  only the assigned operator decides [only the assigned operator]
PASS  a case assigned to one operator cannot be taken by another [assigned to another operator]
PASS  FR-DSP-06 every decision has a reason [dispute_decision_fields]
PASS  FR-PAY-11 a partial refund is above zero and below the item price [dispute_partial_amount]
PASS  FR-DSP-06(d) refund after return needs the fare and who pays it [dispute_return_fields]
PASS  FR-DSP-06 a partial refund is decided
PASS  FR-DSP-09 the history shows open, under review and decided, each with its actor
PASS  FR-TNS-04 losing a dispute is a seller strike
PASS  a decided dispute is closed [dispute is closed]
PASS  the dispute history is append-only [append-only]
PASS  CP-30 a decision on BDT 20,000 or more needs four-eyes approval [four-eyes approval]
PASS  CP-30 with an executed approval the large decision goes through
PASS  FR-DSP-06(d) refund after return waits for the item
PASS  no new evidence once the decision is made [only while the dispute is being considered]
PASS  ... and is decided when the item is back
PASS  only the buyer withdraws [only the buyer]
PASS  FR-DSP-10 the buyer withdraws before a decision; an event tells Unit 2
PASS  FR-DSP-10 a withdrawn dispute cannot be reopened with the same type [cannot be reopened]
PASS  ... but another type can be opened
PASS  a decided dispute cannot be withdrawn [cannot be withdrawn]
PASS  FR-DSP-04 after CP-20 the seller can no longer respond [response window has ended]
PASS  FR-DSP-04 when CP-20 passes, the review starts on the evidence available
PASS  the operator can ask the seller for more information (awaiting_seller)
PASS  ... and the answer returns it to review
PASS  two strikes within 90 days do not yet suspend
PASS  the same strike for the same order counts once
PASS  FR-TNS-04 the CP-47 third strike suspends selling automatically, until review
PASS  a fourth strike does not add a second suspension
PASS  FR-TNS-05 a selling suspension stops selling but not buying
PASS  strikes are append-only [append-only]
PASS  FR-TNS-03 every sanction has a reason [sanctions_reason_check]
PASS  a warning has no duration [sanction_warning_instant]
PASS  a listing removal names the listing [sanction_listing]
PASS  a sanction by an operator names the operator [sanction_actor]
PASS  FR-TNS-05 an account suspension for fraud stops everything and holds payouts
PASS  an expired sanction has no effect
PASS  a sanction cannot be rewritten [cannot be rewritten]
PASS  CP-30 lifting an account suspension needs four-eyes approval [sanction_account_lift_approved]
PASS  with approval the suspension is lifted
PASS  a lifted sanction is final [already lifted]
PASS  sanctions are never deleted [append-only]
PASS  a user cannot report themselves [report_not_self]
```

```
PASS  a reported message names the message [report_message_fields]
PASS  one open report per reporter and target [reports_one_open]
PASS  a resolved report records who resolved it and why [report_resolved_fields]
PASS  a signal delivered twice is stored once [signal_once]
PASS  FR-REV-02 no rating for an order that never finished [ratings_order_id_fkey]
PASS  only the two parties of the order rate each other [only the buyer and the seller]
PASS  FR-REV-01 buyer and seller each rate the other
PASS  FR-REV-02 at most one rating in each direction per order [rating_once]
PASS  FR-REV-01 no rating after the CP-39 window [rating window has ended]
PASS  FR-REV-03 a rating may be edited within its edit period
PASS  FR-REV-03 and fixed afterwards [fixed after its edit period]
PASS  FR-REV-06 a reply needs a written review to reply to [rating_reply_fields]
PASS  FR-REV-06 one reply, not changed later [only one reply]
PASS  FR-REV-07 a removal records the operator and a reason [rating_removal_fields]
PASS  FR-REV-04, FR-REV-07 removed ratings leave the average; completed orders counted
PASS  the seeded SRS defaults satisfy every rule
PASS  FR-OPS-08 a change is a new version; the old one is kept unchanged
PASS  FR-OPS-09 CP-19 not longer than CP-18 is refused [must be longer than the CP-18]
PASS  FR-OPS-09 CP-45 outside 3 to 10 is refused [CP-45 must be 3-10]
PASS  FR-OPS-09 the four-eyes list may only grow [may only grow]
PASS  FR-OPS-09 all broken rules are reported [at least 3 days; CP-29]
PASS  SEC-IRM-04 an emergency switch changes only with four-eyes approval [four-eyes approval]
PASS  FR-GAM-01 turning the game-account feature off also needs it [four-eyes approval]
PASS  SEC-IRM-04 with approval the switch changes
PASS  an unknown parameter is refused [unknown parameter]
PASS  parameter versions are append-only [append-only]
PASS  FR-PRV-06 nothing is disclosed without four-eyes approval [disclosure_needs_approval]
PASS  FR-PRV-06 the user is told, or the reason not to is recorded [disclosure_notice]
PASS  FR-PRV-03 one data export in preparation per user [export_one_pending]
PASS  application role cannot rewrite dispute history [permission denied]
PASS  application role cannot delete sanctions [permission denied]
PASS  application role cannot rewrite parameter versions [permission denied]

87 checks passed (07_disputes_trust_staff_tests.sql, PostgreSQL 16.15 with the Unit 1-6 schemas loaded).
The Unit 1-6 test scripts still pass with Unit 7 loaded (46, 45, 67, 93, 70 and 69 checks).

Concurrency (07_disputes_trust_staff_concurrency.sh). Three runs on fresh databases gave the
same results; only the split in the decide-or-withdraw race varied (9/11, 9/11 and 12/8).
  Opening race: 10 sessions open a dispute on the same order at once (different types)
    results: opened 1, refused 9
    database: disputes on the order=1
  Decide-or-withdraw race: on 20 disputes, the operator decides while the buyer withdraws
    results: decided 9, withdrawn 11
    database: decided=9, withdrawn=11, other=0
  Assignment race: 10 operators take the same unassigned dispute at once
    results: refused 9 of 10
    database: assigned to one operator=true
  Evidence race: 15 uploads at once to one dispute (CP-33 limit 10)
    results: stored 10, refused 5
  Strike race: 10 strikes for one seller at once (CP-47 = 3 in 90 days)
    database: strikes=10, automatic suspensions=1
  Parameter race: 10 administrators publish changes at once
    database: versions=11 (1..11), each with 49-49 values
```

B.2 Schema

```
-- ================================================================
-- TrustBazaar - Unit 7: Disputes, Trust & Safety, Staff Tools - PostgreSQL 15 schema
-- Design document: docs/design/07_disputes_trust_staff.pdf
-- Requirements: SRS v2.0 FR-DSP-01..12, FR-TNS-01..09, FR-REV-01..07, FR-OPS-01..09,
--   FR-PRV-03, FR-PRV-06, FR-GAM-18 (sanction side); SEC-LOG-01 (case views), SEC-IRM-04.
-- Depends on: 01_identity_access.sql (identity.users, identity.staff_accounts,
--   identity.approvals, identity.forbid_change), 02_orders_payments.sql (orders.orders).
-- Conventions DC-1..DC-14. No function reads another unit's tables (DC-1); facts such as
-- "the dispute window is open" arrive as arguments from the owning unit's port.
-- ================================================================

CREATE SCHEMA IF NOT EXISTS disputes;
CREATE SCHEMA IF NOT EXISTS trust;
CREATE SCHEMA IF NOT EXISTS staff;

-- ================================================================
-- DISPUTES (FR-DSP)
```

```
-- ======================================================================
CREATE TYPE disputes.dispute_type AS ENUM
  ('not_as_described', 'damaged', 'not_received', 'credentials_invalid', 'seller_recovery', 'other');   -- FR
     -DSP-01
CREATE TYPE disputes.status AS ENUM
  ('open', 'awaiting_seller', 'under_review', 'awaiting_return', 'decided', 'withdrawn');               -- FR
     -DSP-09
CREATE TYPE disputes.outcome AS ENUM ('release', 'full_refund', 'partial_refund', 'refund_after_return'); --
     FR-DSP-06
CREATE TYPE disputes.party AS ENUM ('buyer', 'seller');
CREATE TYPE disputes.actor AS ENUM ('buyer', 'seller', 'operator', 'system');

CREATE TABLE disputes.disputes (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id            uuid NOT NULL REFERENCES orders.orders(id),
  buyer_id            uuid NOT NULL REFERENCES identity.users(id),
  seller_id           uuid NOT NULL REFERENCES identity.users(id),
  item_kind           varchar(12) NOT NULL CHECK (item_kind IN ('physical', 'game_account')),
  type                disputes.dispute_type NOT NULL,
  description         varchar(4000) NOT NULL CHECK (length(btrim(description)) >= 20),
  item_price          bigint NOT NULL CHECK (item_price > 0),        -- from the order, for partial refunds
  amount_collected    bigint NOT NULL CHECK (amount_collected >= item_price),   -- for the CP-30 threshold
  status              disputes.status NOT NULL DEFAULT 'open',
  opened_by           disputes.actor NOT NULL DEFAULT 'buyer',       -- system: re-key overdue (Unit 5)
  opened_at           timestamptz NOT NULL DEFAULT now(),
  seller_due_at       timestamptz NOT NULL,                          -- opened_at + CP-20
  info_due_at         timestamptz,                                   -- awaiting_seller: operator's request
  review_started_at   timestamptz,
  decision_target_at  timestamptz,                                   -- review start + CP-21
  assigned_to         uuid REFERENCES identity.staff_accounts(id),
  outcome             disputes.outcome,
  item_refund         bigint,                                        -- partial refund (FR-PAY-11)
  return_fare         bigint,
  return_fare_payer   disputes.party,
  decision_reason     text,
  decided_by          uuid REFERENCES identity.staff_accounts(id),
  decided_at          timestamptz,
  approval_id         uuid REFERENCES identity.approvals(id),        -- four-eyes (CP-30)
  closed_at           timestamptz,
  withdrawn_at        timestamptz,
  last_actor          disputes.actor NOT NULL DEFAULT 'buyer',
  last_actor_id       uuid,
  last_reason         text,
  CONSTRAINT dispute_parties CHECK (buyer_id <> seller_id),
  -- game types only for game accounts, physical types only for physical items (FR-DSP-01)
  CONSTRAINT dispute_type_fits_item CHECK (
         (item_kind = 'game_account' AND type IN ('credentials_invalid', 'seller_recovery', 'other'))
      OR (item_kind = 'physical' AND type IN ('not_as_described', 'damaged', 'not_received', 'other'))),
  CONSTRAINT dispute_review_fields CHECK (status IN ('open', 'withdrawn') OR (review_started_at IS NOT NULL
     AND decision_target_at IS NOT NULL)),
  CONSTRAINT dispute_info_fields CHECK ((status = 'awaiting_seller') = (info_due_at IS NOT NULL)),
  CONSTRAINT dispute_decision_fields CHECK ((status IN ('awaiting_return', 'decided')) =
         (outcome IS NOT NULL AND decided_by IS NOT NULL AND decided_at IS NOT NULL AND length(btrim(coalesce
     (decision_reason, ''))) >= 20)),
  CONSTRAINT dispute_partial_amount CHECK ((outcome = 'partial_refund') = (item_refund IS NOT NULL)
         AND (item_refund IS NULL OR (item_refund > 0 AND item_refund < item_price))),
  CONSTRAINT dispute_return_fields CHECK ((outcome = 'refund_after_return') = (return_fare IS NOT NULL AND
     return_fare_payer IS NOT NULL)
         AND (return_fare IS NULL OR return_fare >= 0)),
  CONSTRAINT dispute_return_status CHECK (status <> 'awaiting_return' OR outcome = 'refund_after_return'),
  CONSTRAINT dispute_closed_fields CHECK ((status = 'decided') = (closed_at IS NOT NULL)),
  CONSTRAINT dispute_withdrawn_fields CHECK ((status = 'withdrawn') = (withdrawn_at IS NOT NULL))
);
-- FR-DSP-11: one dispute open on an order at a time
CREATE UNIQUE INDEX disputes_one_open ON disputes.disputes (order_id) WHERE status NOT IN ('decided', '
     withdrawn');
CREATE INDEX disputes_queue ON disputes.disputes (opened_at) WHERE status NOT IN ('decided', 'withdrawn');
CREATE INDEX disputes_seller_due ON disputes.disputes (seller_due_at) WHERE status = 'open';

CREATE FUNCTION disputes.dispute_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    IF NEW.status <> 'open' THEN RAISE EXCEPTION 'a␣dispute␣starts␣as␣open' USING ERRCODE = 'check_violation'
     ; END IF;
    -- FR-DSP-10: a withdrawn dispute cannot be reopened with the same type
```

```
    IF EXISTS (SELECT 1 FROM disputes.disputes WHERE order_id = NEW.order_id AND type = NEW.type AND status =
      'withdrawn') THEN
      RAISE EXCEPTION 'a dispute of this type was withdrawn and cannot be reopened' USING ERRCODE = '
     check_violation';
    END IF;
    RETURN NEW;
  END IF;
  IF (NEW.order_id, NEW.buyer_id, NEW.seller_id, NEW.item_kind, NEW.type, NEW.description, NEW.item_price,
     NEW.amount_collected, NEW.opened_at, NEW.opened_by)
      IS DISTINCT FROM (OLD.order_id, OLD.buyer_id, OLD.seller_id, OLD.item_kind, OLD.type, OLD.description,
     OLD.item_price, OLD.amount_collected, OLD.opened_at, OLD.opened_by) THEN
    RAISE EXCEPTION 'what a dispute is about is fixed' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.status IN ('decided', 'withdrawn') THEN RAISE EXCEPTION 'dispute is closed' USING ERRCODE = '
     check_violation'; END IF;
  IF OLD.outcome IS NOT NULL AND (NEW.outcome, NEW.item_refund, NEW.return_fare, NEW.return_fare_payer, NEW.
     decided_by, NEW.decided_at, NEW.decision_reason)
       IS DISTINCT FROM (OLD.outcome, OLD.item_refund, OLD.return_fare, OLD.return_fare_payer, OLD.decided_by
     , OLD.decided_at, OLD.decision_reason) THEN
    RAISE EXCEPTION 'a decision is final' USING ERRCODE = 'check_violation';
  END IF;
  IF NEW.status IS DISTINCT FROM OLD.status AND NOT ((OLD.status, NEW.status) IN (
       ('open'::disputes.status, 'under_review'::disputes.status), ('open', 'withdrawn'),
       ('under_review', 'awaiting_seller'), ('under_review', 'decided'), ('under_review', 'awaiting_return'),
      ('under_review', 'withdrawn'),
       ('awaiting_seller', 'under_review'), ('awaiting_seller', 'withdrawn'),
       ('awaiting_return', 'decided'))) THEN
    RAISE EXCEPTION 'dispute % -> % is not permitted', OLD.status, NEW.status USING ERRCODE = '
     check_violation';
  END IF;
  -- FR-DSP-06/CP-30: a decision on BDT 20,000 or more needs an executed four-eyes approval
  IF NEW.outcome IS NOT NULL AND OLD.outcome IS NULL AND NEW.amount_collected >= 2000000 AND NEW.approval_id
     IS NULL THEN
    RAISE EXCEPTION 'decisions on BDT 20,000 or more need four-eyes approval (CP-30)' USING ERRCODE = '
     check_violation';
  END IF;
  IF NEW.status IS DISTINCT FROM OLD.status AND (NEW.last_actor_id IS NOT DISTINCT FROM OLD.last_actor_id
       AND NEW.last_actor IS NOT DISTINCT FROM OLD.last_actor AND NEW.last_reason IS NOT DISTINCT FROM OLD.
     last_reason
       AND NEW.last_actor <> 'system') THEN
    RAISE EXCEPTION 'every status change names its actor and reason (FR-DSP-09)' USING ERRCODE = '
     check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER dispute_guard BEFORE INSERT OR UPDATE ON disputes.disputes FOR EACH ROW EXECUTE FUNCTION
     disputes.dispute_guard();

CREATE TABLE disputes.history (                                          -- FR-DSP-09, append-only
  id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dispute_id   uuid NOT NULL REFERENCES disputes.disputes(id),
  from_status  disputes.status,
  to_status    disputes.status NOT NULL,
  actor        disputes.actor NOT NULL,
  actor_id     uuid,
  reason       text,
  at           timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX history_dispute ON disputes.history (dispute_id, id);
CREATE TRIGGER history_append_only BEFORE UPDATE OR DELETE ON disputes.history FOR EACH ROW EXECUTE FUNCTION
     identity.forbid_change();

CREATE FUNCTION disputes.log_status() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' OR NEW.status IS DISTINCT FROM OLD.status THEN
    INSERT INTO disputes.history (dispute_id, from_status, to_status, actor, actor_id, reason)
    VALUES (NEW.id, CASE WHEN TG_OP = 'UPDATE' THEN OLD.status END, NEW.status, NEW.last_actor, NEW.
     last_actor_id, NEW.last_reason);
  END IF;
  RETURN NULL;
END $$;
CREATE TRIGGER dispute_log AFTER INSERT OR UPDATE ON disputes.disputes FOR EACH ROW EXECUTE FUNCTION disputes
     .log_status();

-- statements and evidence of both parties (FR-DSP-01, FR-DSP-04); append-only
```

```
CREATE TABLE disputes.statements (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dispute_id  uuid NOT NULL REFERENCES disputes.disputes(id),
  party       disputes.party NOT NULL,
  text        varchar(4000) NOT NULL CHECK (length(btrim(text)) >= 1),
  at          timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE disputes.evidence (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  dispute_id   uuid NOT NULL REFERENCES disputes.disputes(id),
  party        disputes.party NOT NULL,
  storage_key  varchar(80) NOT NULL UNIQUE,        -- Unit 1 FileVault, evidence purpose (encrypted)
  sha256       bytea NOT NULL CHECK (octet_length(sha256) = 32),
  mime         varchar(30) NOT NULL CHECK (mime IN ('image/jpeg', 'image/png', 'image/webp', 'application/pdf
    ')),
  byte_size    integer NOT NULL CHECK (byte_size BETWEEN 1 AND 10485760),     -- CP-33
  at           timestamptz NOT NULL DEFAULT now()
);
CREATE TRIGGER statements_append_only BEFORE UPDATE OR DELETE ON disputes.statements FOR EACH ROW EXECUTE
    FUNCTION identity.forbid_change();
CREATE TRIGGER evidence_append_only BEFORE UPDATE OR DELETE ON disputes.evidence FOR EACH ROW EXECUTE
    FUNCTION identity.forbid_change();

CREATE FUNCTION disputes.evidence_limit() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE st disputes.status;
BEGIN
  SELECT status INTO st FROM disputes.disputes WHERE id = NEW.dispute_id FOR UPDATE;
  IF st IN ('decided', 'withdrawn', 'awaiting_return') THEN
    RAISE EXCEPTION 'evidence␣is␣accepted␣only␣while␣the␣dispute␣is␣being␣considered' USING ERRCODE = '
    check_violation';
  END IF;
  IF (SELECT count(*) FROM disputes.evidence WHERE dispute_id = NEW.dispute_id) >= 10 THEN
    RAISE EXCEPTION 'at␣most␣10␣evidence␣files␣per␣dispute␣(CP-33)' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER evidence_limit BEFORE INSERT ON disputes.evidence FOR EACH ROW EXECUTE FUNCTION disputes.
    evidence_limit();

CREATE TABLE disputes.outbox (
  id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type      varchar(80) NOT NULL CHECK (event_type ~ '^(disputes|trust|staff)\.[a-z_]+\.[a-z_]+$'),
  aggregate_id    uuid NOT NULL,
  payload         jsonb NOT NULL,
  created_at      timestamptz NOT NULL DEFAULT now(),
  attempts        smallint NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  dispatched_at   timestamptz
);
CREATE INDEX unit7_outbox_pending ON disputes.outbox (next_attempt_at) WHERE dispatched_at IS NULL;

-- ------------------------------------------------------------------ dispute operations
-- FR-DSP-01: the window check (Unit 2 EscrowQuery.disputeAllowed, Unit 4 notReceivedAllowed,
-- Unit 5 credentialsInvalidAllowed) is passed in; the service then freezes the escrow in the
-- same transaction (DC-13).
CREATE FUNCTION disputes.open(p_order uuid, p_buyer uuid, p_seller uuid, p_item_kind varchar, p_type disputes
    .dispute_type,
                              p_description text, p_item_price bigint, p_collected bigint, p_window_ok
    boolean,
                              p_seller_window interval, p_by disputes.actor DEFAULT 'buyer')
RETURNS uuid LANGUAGE plpgsql AS $$
DECLARE did uuid;
BEGIN
  IF NOT p_window_ok THEN
    RAISE EXCEPTION 'a␣dispute␣of␣this␣type␣cannot␣be␣opened␣now␣(outside␣its␣window,␣or␣the␣payment␣is␣not␣
    held)'
      USING ERRCODE = 'check_violation';
  END IF;
  BEGIN
    INSERT INTO disputes.disputes (order_id, buyer_id, seller_id, item_kind, type, description, item_price,
    amount_collected,
                                   seller_due_at, opened_by, last_actor, last_actor_id, last_reason)
    VALUES (p_order, p_buyer, p_seller, p_item_kind, p_type, p_description, p_item_price, p_collected,
            now() + p_seller_window, p_by, p_by, CASE WHEN p_by = 'buyer' THEN p_buyer END, 'opened')
    RETURNING id INTO did;
```

```
  EXCEPTION WHEN unique_violation THEN
    RAISE EXCEPTION 'a␣dispute␣is␣already␣open␣on␣this␣order␣(FR-DSP-11)' USING ERRCODE = 'check_violation';
  END;
  INSERT INTO disputes.outbox (event_type, aggregate_id, payload)
  VALUES ('disputes.dispute.opened', did, jsonb_build_object('orderId', p_order, 'type', p_type, 'sellerDueAt
     ', now() + p_seller_window));
  RETURN did;
END $$;

-- FR-DSP-04: the seller responds within CP-20 (or within the operator's request)
CREATE FUNCTION disputes.seller_respond(p_dispute uuid, p_seller uuid, p_text text, p_review_target interval)
      RETURNS void LANGUAGE plpgsql AS $$
DECLARE d disputes.disputes;
BEGIN
  SELECT * INTO d FROM disputes.disputes WHERE id = p_dispute FOR UPDATE;
  IF NOT FOUND OR p_seller <> d.seller_id THEN RAISE EXCEPTION 'only␣the␣seller␣responds' USING ERRCODE = '
     insufficient_privilege'; END IF;
  IF d.status NOT IN ('open', 'awaiting_seller') THEN RAISE EXCEPTION 'no␣response␣is␣expected␣now' USING
     ERRCODE = 'check_violation'; END IF;
  IF now() >= coalesce(d.info_due_at, d.seller_due_at) THEN
    RAISE EXCEPTION 'the␣response␣window␣has␣ended;␣the␣operator␣decides␣on␣the␣evidence␣available' USING
     ERRCODE = 'check_violation';
  END IF;
  INSERT INTO disputes.statements (dispute_id, party, text) VALUES (p_dispute, 'seller', p_text);
  UPDATE disputes.disputes SET status = 'under_review', info_due_at = NULL,
         review_started_at = coalesce(review_started_at, now()),
         decision_target_at = coalesce(decision_target_at, now() + p_review_target),
         last_actor = 'seller', last_actor_id = p_seller, last_reason = 'seller␣responded'
   WHERE id = p_dispute;
END $$;

-- the CP-20 job: no response in time, so the review starts on the evidence available
CREATE FUNCTION disputes.seller_window_ended(p_dispute uuid, p_review_target interval) RETURNS boolean
     LANGUAGE plpgsql AS $$
BEGIN
  UPDATE disputes.disputes SET status = 'under_review', info_due_at = NULL,
         review_started_at = coalesce(review_started_at, now()), decision_target_at = coalesce(
    decision_target_at, now() + p_review_target),
         last_actor = 'system', last_actor_id = NULL, last_reason = 'response␣window␣ended'
   WHERE id = p_dispute AND ((status = 'open' AND now() >= seller_due_at) OR (status = 'awaiting_seller' AND
     now() >= info_due_at));
  RETURN FOUND;
END $$;

-- the operator asks the seller for more information (FR-DSP-09 awaiting_seller)
CREATE FUNCTION disputes.request_info(p_dispute uuid, p_staff uuid, p_due interval, p_note text) RETURNS void
      LANGUAGE plpgsql AS $$
BEGIN
  UPDATE disputes.disputes SET status = 'awaiting_seller', info_due_at = now() + p_due,
         last_actor = 'operator', last_actor_id = p_staff, last_reason = p_note
   WHERE id = p_dispute AND status = 'under_review' AND assigned_to = p_staff;
  IF NOT FOUND THEN RAISE EXCEPTION 'only␣the␣assigned␣operator␣can␣ask,␣during␣review' USING ERRCODE = '
     check_violation'; END IF;
END $$;

-- an operator takes the case (the conflict-of-interest check of FR-OPS-06 is Unit 1's, at claim)
CREATE FUNCTION disputes.assign(p_dispute uuid, p_staff uuid) RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  UPDATE disputes.disputes SET assigned_to = p_staff WHERE id = p_dispute AND status NOT IN ('decided', '
     withdrawn')
      AND (assigned_to IS NULL OR assigned_to = p_staff);
  IF NOT FOUND THEN RAISE EXCEPTION 'dispute␣is␣closed␣or␣assigned␣to␣another␣operator' USING ERRCODE = '
     check_violation'; END IF;
END $$;

-- FR-DSP-10: the buyer withdraws before a decision
CREATE FUNCTION disputes.withdraw(p_dispute uuid, p_buyer uuid) RETURNS void LANGUAGE plpgsql AS $$
DECLARE d disputes.disputes;
BEGIN
  SELECT * INTO d FROM disputes.disputes WHERE id = p_dispute FOR UPDATE;
  IF NOT FOUND OR p_buyer <> d.buyer_id THEN RAISE EXCEPTION 'only␣the␣buyer␣withdraws' USING ERRCODE = '
     insufficient_privilege'; END IF;
  IF d.status NOT IN ('open', 'under_review', 'awaiting_seller') THEN
    RAISE EXCEPTION 'a␣decided␣dispute␣cannot␣be␣withdrawn' USING ERRCODE = 'check_violation';
  END IF;
```

```
  UPDATE disputes.disputes SET status = 'withdrawn', withdrawn_at = now(), info_due_at = NULL,
         last_actor = 'buyer', last_actor_id = p_buyer, last_reason = 'withdrawn␣by␣the␣buyer'
   WHERE id = p_dispute;
  INSERT INTO disputes.outbox (event_type, aggregate_id, payload)
  VALUES ('disputes.dispute.withdrawn', p_dispute, jsonb_build_object('orderId', d.order_id));
END $$;

-- FR-DSP-06: the assigned operator decides; the service applies the outcome through Unit 2 in the
-- same transaction (or books the return through Unit 4 for outcome (d)). Losing a dispute is a
-- seller strike (FR-TNS-04).
CREATE FUNCTION disputes.decide(p_dispute uuid, p_staff uuid, p_outcome disputes.outcome, p_reason text,
                                p_item_refund bigint DEFAULT NULL, p_fare bigint DEFAULT NULL, p_payer
    disputes.party DEFAULT NULL,
                                p_approval uuid DEFAULT NULL, p_strike_window interval DEFAULT '90␣days',
    p_strike_limit integer DEFAULT 3)
RETURNS disputes.status LANGUAGE plpgsql AS $$
DECLARE d disputes.disputes; st disputes.status;
BEGIN
  SELECT * INTO d FROM disputes.disputes WHERE id = p_dispute FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'no␣such␣dispute' USING ERRCODE = 'check_violation'; END IF;
  IF d.assigned_to IS DISTINCT FROM p_staff THEN
    RAISE EXCEPTION 'only␣the␣assigned␣operator␣decides' USING ERRCODE = 'insufficient_privilege';
  END IF;
  IF d.status <> 'under_review' THEN RAISE EXCEPTION 'dispute␣is␣not␣under␣review' USING ERRCODE = '
    check_violation'; END IF;
  st := CASE WHEN p_outcome = 'refund_after_return' THEN 'awaiting_return'::disputes.status ELSE 'decided'
    END;
  UPDATE disputes.disputes
     SET status = st, outcome = p_outcome, item_refund = p_item_refund, return_fare = p_fare,
     return_fare_payer = p_payer,
         decision_reason = p_reason, decided_by = p_staff, decided_at = now(), approval_id = p_approval,
         closed_at = CASE WHEN st = 'decided' THEN now() END,
         last_actor = 'operator', last_actor_id = p_staff, last_reason = p_reason
   WHERE id = p_dispute;
  IF p_outcome <> 'release' THEN
    PERFORM trust.record_strike(d.seller_id, 'dispute_lost', d.order_id, p_strike_window, p_strike_limit);
  END IF;
  INSERT INTO disputes.outbox (event_type, aggregate_id, payload)
  VALUES ('disputes.dispute.decided', p_dispute, jsonb_build_object('orderId', d.order_id, 'outcome',
    p_outcome, 'status', st));
  RETURN st;
END $$;

-- outcome (d): the item is back with the seller (courier or seller's confirmation)
CREATE FUNCTION disputes.return_completed(p_dispute uuid) RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  UPDATE disputes.disputes SET status = 'decided', closed_at = now(),
         last_actor = 'system', last_actor_id = NULL, last_reason = 'item␣returned␣to␣the␣seller'
   WHERE id = p_dispute AND status = 'awaiting_return';
  IF NOT FOUND THEN RAISE EXCEPTION 'dispute␣is␣not␣awaiting␣a␣return' USING ERRCODE = 'check_violation'; END
     IF;
END $$;

-- ======================================================================
-- TRUST & SAFETY (FR-TNS, FR-REV)
-- ======================================================================
CREATE TYPE trust.report_target   AS ENUM ('listing', 'user', 'message', 'thread');
CREATE TYPE trust.report_category AS ENUM ('fraudulent', 'counterfeit', 'prohibited', 'misrepresented', '
    abusive');   -- FR-TNS-01
CREATE TYPE trust.report_status   AS ENUM ('open', 'under_review', 'actioned', 'dismissed');
CREATE TYPE trust.sanction_kind   AS ENUM ('warning', 'listing_removal', 'selling_suspended', '
    buying_suspended', 'account_suspended');  -- FR-TNS-03
CREATE TYPE trust.strike_reason   AS ENUM ('unilateral_cancel', 'missed_deadline', 'dispute_lost');
              -- FR-TNS-04
CREATE TYPE trust.signal_status   AS ENUM ('open', 'dismissed', 'actioned');

CREATE TABLE trust.reports (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  reporter_id   uuid NOT NULL REFERENCES identity.users(id),
  target_type   trust.report_target NOT NULL,
  target_id     uuid NOT NULL,                       -- the listing, user or thread (no FK across units, DC
    -5)
  message_id    bigint,                              -- for a reported message (Unit 6)
  subject_user  uuid NOT NULL REFERENCES identity.users(id),   -- whom the report is about
  category      trust.report_category NOT NULL,
```

```
  text          varchar(2000),
  status        trust.report_status NOT NULL DEFAULT 'open',
  assigned_to   uuid REFERENCES identity.staff_accounts(id),
  resolution    text,
  resolved_by   uuid REFERENCES identity.staff_accounts(id),
  resolved_at   timestamptz,
  created_at    timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT report_not_self CHECK (reporter_id <> subject_user),
  CONSTRAINT report_message_fields CHECK ((target_type = 'message') = (message_id IS NOT NULL)),
  CONSTRAINT report_resolved_fields CHECK ((status IN ('actioned', 'dismissed')) =
         (resolved_at IS NOT NULL AND resolved_by IS NOT NULL AND length(btrim(coalesce(resolution, ''))) >=
    10))
);
-- one open report per reporter and target
CREATE UNIQUE INDEX reports_one_open ON trust.reports (reporter_id, target_type, target_id, coalesce(
    message_id, 0))
  WHERE status IN ('open', 'under_review');
CREATE INDEX reports_queue ON trust.reports (created_at) WHERE status IN ('open', 'under_review');

CREATE TABLE trust.sanctions (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          uuid NOT NULL REFERENCES identity.users(id),
  kind             trust.sanction_kind NOT NULL,
  reason           text NOT NULL CHECK (length(btrim(reason)) >= 10),
  fraud            boolean NOT NULL DEFAULT false,        -- FR-TNS-05: holds payouts
  starts_at        timestamptz NOT NULL DEFAULT now(),
  ends_at          timestamptz,                           -- NULL: until review
  automatic        boolean NOT NULL DEFAULT false,        -- CP-47 strikes
  created_by       uuid REFERENCES identity.staff_accounts(id),
  listing_id       uuid,                                  -- listing_removal (Unit 3; no FK, DC-5 permits one
     but the listing may be purged)
  related_type     varchar(20) CHECK (related_type IN ('report', 'dispute', 'signal', 'strikes', 'gam18')),
  related_id       uuid,
  lifted_at        timestamptz,
  lifted_by        uuid REFERENCES identity.staff_accounts(id),
  lift_reason      text,
  lift_approval_id uuid REFERENCES identity.approvals(id),
  CONSTRAINT sanction_actor CHECK (automatic OR created_by IS NOT NULL),
  CONSTRAINT sanction_period CHECK (ends_at IS NULL OR ends_at > starts_at),
  CONSTRAINT sanction_warning_instant CHECK (kind NOT IN ('warning', 'listing_removal') OR ends_at IS NULL),
  CONSTRAINT sanction_listing CHECK ((kind = 'listing_removal') = (listing_id IS NOT NULL)),
  CONSTRAINT sanction_lift_fields CHECK ((lifted_at IS NULL) = (lifted_by IS NULL)
         AND (lifted_at IS NULL OR length(btrim(coalesce(lift_reason, ''))) >= 10)),
  -- CP-30: lifting an account suspension needs four-eyes approval
  CONSTRAINT sanction_account_lift_approved CHECK (kind <> 'account_suspended' OR lifted_at IS NULL OR
    lift_approval_id IS NOT NULL)
);
CREATE INDEX sanctions_user_live ON trust.sanctions (user_id) WHERE lifted_at IS NULL;
CREATE UNIQUE INDEX sanctions_one_auto_live ON trust.sanctions (user_id) WHERE automatic AND lifted_at IS
    NULL;

CREATE FUNCTION trust.sanction_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF (NEW.user_id, NEW.kind, NEW.reason, NEW.fraud, NEW.starts_at, NEW.automatic, NEW.created_by, NEW.
    listing_id)
     IS DISTINCT FROM (OLD.user_id, OLD.kind, OLD.reason, OLD.fraud, OLD.starts_at, OLD.automatic, OLD.
    created_by, OLD.listing_id) THEN
    RAISE EXCEPTION 'a␣sanction␣cannot␣be␣rewritten;␣lift␣it␣and␣apply␣a␣new␣one' USING ERRCODE = '
    check_violation';
  END IF;
  IF OLD.lifted_at IS NOT NULL THEN RAISE EXCEPTION 'sanction␣already␣lifted' USING ERRCODE = '
    check_violation'; END IF;
  IF NEW.ends_at IS DISTINCT FROM OLD.ends_at AND NEW.lifted_at IS NULL THEN
    RAISE EXCEPTION 'change␣a␣sanction''s␣end␣only␣by␣lifting␣it' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER sanction_guard BEFORE UPDATE ON trust.sanctions FOR EACH ROW EXECUTE FUNCTION trust.
    sanction_guard();
CREATE TRIGGER sanction_no_delete BEFORE DELETE ON trust.sanctions FOR EACH ROW EXECUTE FUNCTION identity.
    forbid_change();

-- what a user may do now (SanctionPort, used by Unit 1's assertCanTransact and by Unit 2's payouts)
CREATE FUNCTION trust.restrictions(p_user uuid, OUT can_sell boolean, OUT can_buy_bid boolean, OUT fraud_hold
     boolean)
```

```
LANGUAGE sql STABLE AS $$
  SELECT NOT bool_or(kind IN ('selling_suspended', 'account_suspended')) IS TRUE,
         NOT bool_or(kind IN ('buying_suspended', 'account_suspended')) IS TRUE,
         bool_or(fraud) IS TRUE
    FROM trust.sanctions
   WHERE user_id = p_user AND lifted_at IS NULL AND kind NOT IN ('warning', 'listing_removal')
     AND starts_at <= now() AND (ends_at IS NULL OR ends_at > now())
$$;

CREATE TABLE trust.strikes (                                                    -- FR-TNS-04
  id        bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id   uuid NOT NULL REFERENCES identity.users(id),
  reason    trust.strike_reason NOT NULL,
  order_id  uuid NOT NULL REFERENCES orders.orders(id),
  at        timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT strike_once UNIQUE (order_id, reason)
);
CREATE INDEX strikes_user ON trust.strikes (user_id, at);
CREATE TRIGGER strikes_append_only BEFORE UPDATE OR DELETE ON trust.strikes FOR EACH ROW EXECUTE FUNCTION
    identity.forbid_change();

-- records a strike; at CP-47 strikes within the window, suspends selling until review (once).
-- Returns true if this strike caused the suspension.
CREATE FUNCTION trust.record_strike(p_user uuid, p_reason trust.strike_reason, p_order uuid,
                                    p_window interval, p_limit integer) RETURNS boolean LANGUAGE plpgsql AS
    $$
DECLARE n integer; inserted integer;
BEGIN
  PERFORM pg_advisory_xact_lock(hashtextextended('trust.strikes:' || p_user::text, 0));
  INSERT INTO trust.strikes (user_id, reason, order_id) VALUES (p_user, p_reason, p_order) ON CONFLICT DO
    NOTHING;
  GET DIAGNOSTICS inserted = ROW_COUNT;
  IF inserted = 0 THEN RETURN false; END IF;
  SELECT count(*) INTO n FROM trust.strikes WHERE user_id = p_user AND at > now() - p_window;
  IF n >= p_limit AND NOT EXISTS (SELECT 1 FROM trust.sanctions WHERE user_id = p_user AND lifted_at IS NULL
                                  AND kind IN ('selling_suspended', 'account_suspended')) THEN
    INSERT INTO trust.sanctions (user_id, kind, reason, automatic, related_type)
    VALUES (p_user, 'selling_suspended', format('%s␣seller␣strikes␣within␣%s;␣pending␣review', n, p_window),
    true, 'strikes');
    INSERT INTO disputes.outbox (event_type, aggregate_id, payload)
    VALUES ('trust.sanction.applied', p_user, jsonb_build_object('kind', 'selling_suspended', 'automatic',
    true));
    RETURN true;
  END IF;
  RETURN false;
END $$;

-- flagged activity queue (FR-OPS-01): signals from Units 3 to 6 and the linked-account check
CREATE TABLE trust.signals (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  kind          varchar(40) NOT NULL,             -- e.g. shill_bidding, relisting, linked_accounts,
    chat_off_platform, delivery_issue
  source_ref    varchar(120) NOT NULL,            -- id in the source unit; dedupes repeated events
  subject_user  uuid REFERENCES identity.users(id),
  detail        jsonb NOT NULL DEFAULT '{}',
  status        trust.signal_status NOT NULL DEFAULT 'open',
  reviewed_by   uuid REFERENCES identity.staff_accounts(id),
  reviewed_at   timestamptz,
  note          text,
  created_at    timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT signal_once UNIQUE (kind, source_ref),
  CONSTRAINT signal_review_fields CHECK ((status = 'open') = (reviewed_at IS NULL) AND (status = 'open' OR
    reviewed_by IS NOT NULL))
);
CREATE INDEX signals_queue ON trust.signals (created_at) WHERE status = 'open';

-- ------------------------------------------------------------------ ratings (FR-REV)
CREATE TABLE trust.rating_eligibility (               -- written when an order's escrow is terminal (Unit 2
    events)
  order_id     uuid PRIMARY KEY REFERENCES orders.orders(id),
  buyer_id     uuid NOT NULL REFERENCES identity.users(id),
  seller_id    uuid NOT NULL REFERENCES identity.users(id),
  closed_at    timestamptz NOT NULL,
  rate_until   timestamptz NOT NULL,                -- closed_at + CP-39 rating window
```

```
  completed    boolean NOT NULL,                    -- released to the seller: counts as a completed
     transaction
  CONSTRAINT eligibility_window CHECK (rate_until > closed_at)
);
CREATE TABLE trust.ratings (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id    uuid NOT NULL REFERENCES trust.rating_eligibility(order_id),   -- FR-REV-02: only paid,
     finished orders
  rater_id    uuid NOT NULL REFERENCES identity.users(id),
  ratee_id    uuid NOT NULL REFERENCES identity.users(id),
  stars       smallint NOT NULL CHECK (stars BETWEEN 1 AND 5),
  text        varchar(1000),
  created_at  timestamptz NOT NULL DEFAULT now(),
  edit_until  timestamptz NOT NULL,                -- created + CP-39 edit period
  edited_at   timestamptz,
  reply_text  varchar(1000),                        -- FR-REV-06: one public reply by the rated user
  reply_at    timestamptz,
  removed_at  timestamptz,                          -- FR-REV-07
  removed_by  uuid REFERENCES identity.staff_accounts(id),
  removal_reason text,
  CONSTRAINT rating_once UNIQUE (order_id, rater_id),
  CONSTRAINT rating_not_self CHECK (rater_id <> ratee_id),
  CONSTRAINT rating_reply_fields CHECK ((reply_text IS NULL) = (reply_at IS NULL) AND (reply_text IS NULL OR
     text IS NOT NULL)),
  CONSTRAINT rating_removal_fields CHECK ((removed_at IS NULL) = (removed_by IS NULL)
         AND (removed_at IS NULL OR length(btrim(coalesce(removal_reason, ''))) >= 10))
);
CREATE INDEX ratings_ratee ON trust.ratings (ratee_id) WHERE removed_at IS NULL;

CREATE FUNCTION trust.rating_guard() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE e trust.rating_eligibility;
BEGIN
  IF TG_OP = 'INSERT' THEN
    SELECT * INTO e FROM trust.rating_eligibility WHERE order_id = NEW.order_id;
    IF (NEW.rater_id, NEW.ratee_id) NOT IN ((e.buyer_id, e.seller_id), (e.seller_id, e.buyer_id)) THEN
      RAISE EXCEPTION 'only␣the␣buyer␣and␣the␣seller␣of␣the␣order␣rate␣each␣other' USING ERRCODE = '
     insufficient_privilege';
    END IF;
    IF now() > e.rate_until THEN                          -- CP-39 rating window
      RAISE EXCEPTION 'the␣rating␣window␣has␣ended␣(CP-39)' USING ERRCODE = 'check_violation';
    END IF;
    IF NEW.reply_text IS NOT NULL OR NEW.removed_at IS NOT NULL THEN
      RAISE EXCEPTION 'a␣new␣rating␣has␣no␣reply␣or␣removal' USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
  END IF;
  IF (NEW.order_id, NEW.rater_id, NEW.ratee_id, NEW.created_at, NEW.edit_until) IS DISTINCT FROM (OLD.
     order_id, OLD.rater_id, OLD.ratee_id, OLD.created_at, OLD.edit_until) THEN
    RAISE EXCEPTION 'rating␣identity␣is␣fixed' USING ERRCODE = 'check_violation';
  END IF;
  IF (NEW.stars, NEW.text) IS DISTINCT FROM (OLD.stars, OLD.text) THEN
    IF now() > OLD.edit_until OR OLD.removed_at IS NOT NULL THEN
      RAISE EXCEPTION 'a␣rating␣is␣fixed␣after␣its␣edit␣period␣(CP-39)' USING ERRCODE = 'check_violation';
    END IF;
    NEW.edited_at := now();
  END IF;
  IF OLD.reply_text IS NOT NULL AND (NEW.reply_text, NEW.reply_at) IS DISTINCT FROM (OLD.reply_text, OLD.
     reply_at) THEN
    RAISE EXCEPTION 'only␣one␣reply,␣which␣cannot␣be␣changed␣(FR-REV-06)' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.removed_at IS NOT NULL AND (NEW.removed_at, NEW.removed_by, NEW.removal_reason) IS DISTINCT FROM (
     OLD.removed_at, OLD.removed_by, OLD.removal_reason) THEN
    RAISE EXCEPTION 'a␣removal␣is␣final' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER rating_guard BEFORE INSERT OR UPDATE ON trust.ratings FOR EACH ROW EXECUTE FUNCTION trust.
     rating_guard();

-- FR-REV-04: shown on profiles and listings; removed ratings do not count (FR-REV-07)
CREATE VIEW trust.profile_summary AS
  WITH users AS (SELECT buyer_id AS id FROM trust.rating_eligibility UNION SELECT seller_id FROM trust.
     rating_eligibility)
  SELECT u.id AS user_id,
```

```
        (SELECT round(avg(r.stars)::numeric, 1) FROM trust.ratings r WHERE r.ratee_id = u.id AND r.
    removed_at IS NULL) AS average_rating,
        (SELECT count(*) FROM trust.ratings r WHERE r.ratee_id = u.id AND r.removed_at IS NULL) AS
    rating_count,
        (SELECT count(*) FROM trust.rating_eligibility e WHERE e.completed AND (e.buyer_id = u.id OR e.
    seller_id = u.id)) AS completed_transactions
   FROM users u;

-- ====================================================================
-- STAFF TOOLS: versioned parameters and switches (FR-OPS-03, FR-OPS-08, FR-OPS-09), disclosures
-- and data exports (FR-PRV-03, FR-PRV-06)
-- ====================================================================
CREATE TABLE staff.config_versions (
  version      integer PRIMARY KEY CHECK (version >= 1),
  created_by   uuid REFERENCES identity.staff_accounts(id),      -- NULL only for the seeded version 1
  reason       text NOT NULL CHECK (length(btrim(reason)) >= 10),
  approval_id  uuid REFERENCES identity.approvals(id),            -- needed when a CP-30 control changes
  created_at   timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT config_version_creator CHECK (version = 1 OR created_by IS NOT NULL)
);
CREATE TABLE staff.config_values (
  version  integer NOT NULL REFERENCES staff.config_versions(version),
  code     varchar(24) NOT NULL CHECK (code ~ '^(CP-[0-9]{2}|SW-[a-z_]+)$'),
  value    jsonb NOT NULL,
  PRIMARY KEY (version, code)
);
CREATE TRIGGER config_versions_append_only BEFORE UPDATE OR DELETE ON staff.config_versions FOR EACH ROW
    EXECUTE FUNCTION identity.forbid_change();
CREATE TRIGGER config_values_append_only BEFORE UPDATE OR DELETE ON staff.config_values FOR EACH ROW EXECUTE
    FUNCTION identity.forbid_change();

CREATE FUNCTION staff.current_config_version() RETURNS integer LANGUAGE sql STABLE AS $$
  SELECT max(version) FROM staff.config_versions
$$;

-- FR-OPS-09: every rule of the SRS parameter table, checked on a complete candidate set.
-- Returns the list of broken rules (empty = valid).
CREATE FUNCTION staff.config_violations(c jsonb, prev jsonb) RETURNS text[] LANGUAGE plpgsql STABLE AS $$
DECLARE v text[] := '{}';
BEGIN
  IF (c #>> '{CP-01,access}')::interval NOT BETWEEN interval '5 minutes' AND interval '30 minutes' THEN v :=
    v || 'CP-01 must be 5-30 minutes'::text; END IF;
  IF (c #>> '{CP-02,idle}')::interval > (c #>> '{CP-02,absolute}')::interval THEN v := v || 'CP-02 idle must
    not exceed absolute'::text; END IF;
  IF (c #>> '{CP-03,idle}')::interval > interval '30 minutes' THEN v := v || 'CP-03 staff idle must be at
    most 30 minutes'::text; END IF;
  IF (c #>> '{CP-05,validity}')::interval > interval '10 minutes' THEN v := v || 'CP-05 OTP validity must be
    at most 10 minutes'::text; END IF;
  IF (c #>> '{CP-06,reset}')::interval > interval '60 minutes' THEN v := v || 'CP-06 reset link must be at
    most 60 minutes'::text; END IF;
  IF EXISTS (SELECT 1 FROM jsonb_array_elements_text(c -> 'CP-07') d WHERE d::interval > interval '72 hours')
     THEN v := v || 'CP-07 durations at most 72 hours'::text; END IF;
  IF (c #>> '{CP-08,bps}')::int <= 0 OR (c #>> '{CP-08,min_poisha}')::bigint <= 0 THEN v := v || 'CP-08
    increment must be positive'::text; END IF;
  IF (c #>> '{CP-09,window}')::interval <= interval '0' OR (c #>> '{CP-09,extension}')::interval <= interval
    '0' THEN v := v || 'CP-09 window and extension must be positive'::text; END IF;
  IF (c #>> '{CP-10}')::interval NOT BETWEEN interval '1 hour' AND interval '72 hours' THEN v := v || 'CP-10
    must be 1-72 hours'::text; END IF;
  IF (c #>> '{CP-11}')::interval NOT BETWEEN interval '1 hour' AND interval '72 hours' THEN v := v || 'CP-11
    must be 1-72 hours'::text; END IF;
  IF (c #>> '{CP-12,strikes}')::int < 1 THEN v := v || 'CP-12 strikes must be at least 1'::text; END IF;
  IF (c #>> '{CP-13}')::interval NOT BETWEEN interval '5 minutes' AND interval '60 minutes' THEN v := v || '
    CP-13 must be 5-60 minutes'::text; END IF;
  IF (c #>> '{CP-14}')::numeric <= 0 THEN v := v || 'CP-14 must be positive'::text; END IF;
  IF (c #>> '{CP-18}')::interval >= (c #>> '{CP-19}')::interval THEN v := v || 'CP-19 auto-release must be
    longer than the CP-18 dispute window'::text; END IF;
  IF (c #>> '{CP-24}')::interval < interval '3 days' THEN v := v || 'CP-24 protection period must be at least
    3 days'::text; END IF;
  IF (c #>> '{CP-25,pro_bps}')::int > (c #>> '{CP-25,standard_bps}')::int THEN v := v || 'CP-25 Pro
    commission must not exceed standard'::text; END IF;
  IF (c #>> '{CP-26}')::int NOT BETWEEN 0 AND 5000 THEN v := v || 'CP-26 delivery margin must be 0-50%'::text
    ; END IF;
  IF (c #>> '{CP-28,pro}')::int < (c #>> '{CP-28,standard}')::int THEN v := v || 'CP-28 Pro limit must be at
    least the standard limit'::text; END IF;
```

```
  IF (c #>> '{CP-29}')::interval < interval '12 hours' THEN v := v || 'CP-29 cooling-off must be at least 12 hours'::text; END IF;
  IF prev IS NOT NULL AND NOT ((c -> 'CP-30') @> (prev -> 'CP-30')) THEN v := v || 'CP-30 four-eyes list may only grow'::text; END IF;
  IF (c #>> '{CP-31}')::interval > interval '15 minutes' THEN v := v || 'CP-31 signed links at most 15 minutes'::text; END IF;
  IF (c #>> '{CP-34}')::int < 1 THEN v := v || 'CP-34 must be at least 1'::text; END IF;
  IF (c #>> '{CP-40}')::interval < interval '1 year' THEN v := v || 'CP-40 security-log retention at least 1 year'::text; END IF;
  IF (c #>> '{CP-41}')::interval > interval '30 days' THEN v := v || 'CP-41 erasure within 30 days'::text; END IF;
  IF (c #>> '{CP-45}')::int NOT BETWEEN 3 AND 10 THEN v := v || 'CP-45 must be 3-10'::text; END IF;
  IF (c #>> '{CP-46}')::interval > interval '60 minutes' THEN v := v || 'CP-46 reconciliation at most 60 minutes'::text; END IF;
  IF (c #>> '{CP-47,strikes}')::int < 1 THEN v := v || 'CP-47 must be at least 1'::text; END IF;
  RETURN v;
END $$;

CREATE FUNCTION staff.config_as_json(p_version integer) RETURNS jsonb LANGUAGE sql STABLE AS $$
  SELECT coalesce(jsonb_object_agg(code, value), '{}') FROM staff.config_values WHERE version = p_version
$$;

-- publish a new version: the previous values with the given changes; refused if any rule breaks,
-- or if a switch or a CP-30 item changes without an approval
CREATE FUNCTION staff.publish_config(p_changes jsonb, p_staff uuid, p_reason text, p_approval uuid DEFAULT NULL)
RETURNS integer LANGUAGE plpgsql AS $$
DECLARE prev integer; old jsonb; nxt jsonb; bad text[]; nv integer; k text;
BEGIN
  PERFORM pg_advisory_xact_lock(hashtextextended('staff.config', 0));
  prev := staff.current_config_version();
  old := staff.config_as_json(prev);
  FOR k IN SELECT jsonb_object_keys(p_changes) LOOP
    IF NOT old ? k THEN RAISE EXCEPTION 'unknown parameter %', k USING ERRCODE = 'check_violation'; END IF;
  END LOOP;
  nxt := old || p_changes;
  bad := staff.config_violations(nxt, old);
  IF cardinality(bad) > 0 THEN
    RAISE EXCEPTION 'parameter change refused: %', array_to_string(bad, '; ') USING ERRCODE = 'check_violation';
  END IF;
  IF p_approval IS NULL AND EXISTS (SELECT 1 FROM jsonb_object_keys(p_changes) k2
                                    WHERE (k2 LIKE 'SW-%' OR k2 = 'CP-30' OR k2 = 'CP-36') AND p_changes -> k2 IS DISTINCT FROM old -> k2) THEN
    RAISE EXCEPTION 'switches, CP-30 and CP-36 change only with four-eyes approval (SEC-IRM-04)' USING ERRCODE = 'check_violation';
  END IF;
  nv := prev + 1;
  INSERT INTO staff.config_versions (version, created_by, reason, approval_id) VALUES (nv, p_staff, p_reason, p_approval);
  INSERT INTO staff.config_values (version, code, value) SELECT nv, key, value FROM jsonb_each(nxt);
  INSERT INTO disputes.outbox (event_type, aggregate_id, payload)
  VALUES ('staff.config.published', gen_random_uuid(), jsonb_build_object('version', nv, 'changed', (SELECT jsonb_agg(x) FROM jsonb_object_keys(p_changes) x)));
  RETURN nv;
END $$;

-- FR-PRV-06: disclosure to an authority, only on a documented lawful request, with four-eyes
CREATE TABLE staff.disclosures (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  subject_user     uuid NOT NULL REFERENCES identity.users(id),
  authority        varchar(200) NOT NULL,
  legal_reference  varchar(300) NOT NULL CHECK (length(btrim(legal_reference)) >= 5),   -- the request's reference
  scope            text NOT NULL CHECK (length(btrim(scope)) >= 10),
  requested_by     uuid NOT NULL REFERENCES identity.staff_accounts(id),
  approval_id      uuid REFERENCES identity.approvals(id),
  disclosed_at     timestamptz,
  user_notified_at timestamptz,
  withheld_reason  text,                                  -- the law forbids telling the user
  created_at       timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT disclosure_needs_approval CHECK (disclosed_at IS NULL OR approval_id IS NOT NULL),
  CONSTRAINT disclosure_notice CHECK (user_notified_at IS NULL OR withheld_reason IS NULL)
);
```

```
-- FR-PRV-03: data exports, one at a time per user
CREATE TABLE staff.export_requests (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       uuid NOT NULL REFERENCES identity.users(id),
  requested_at  timestamptz NOT NULL DEFAULT now(),
  ready_at      timestamptz,
  file_key      varchar(80),
  expires_at    timestamptz,
  CONSTRAINT export_ready_fields CHECK ((ready_at IS NULL) = (file_key IS NULL) AND (ready_at IS NULL) = (
    expires_at IS NULL))
);
CREATE UNIQUE INDEX export_one_pending ON staff.export_requests (user_id) WHERE ready_at IS NULL;

-- ====================================================================
-- Seed: configuration version 1 = the SRS defaults (CP-01..CP-47) and the emergency switches
-- ====================================================================
INSERT INTO staff.config_versions (version, reason) VALUES (1, 'SRS v2.0 default values (Table of
    configurable parameters)');
INSERT INTO staff.config_values (version, code, value) VALUES
 (1, 'CP-01', '{"access":"15 minutes"}'), (1, 'CP-02', '{"idle":"7 days","absolute":"30 days"}'),
 (1, 'CP-03', '{"idle":"15 minutes","absolute":"8 hours"}'),
 (1, 'CP-04', '{"per_account":5,"per_address":50,"window":"15 minutes"}'),
 (1, 'CP-05', '{"validity":"5 minutes","attempts":5,"sends_per_hour":3,"sends_per_day":10}'),
 (1, 'CP-06', '{"reset":"30 minutes","verify":"24 hours"}'),
 (1, 'CP-07', '["1 hour","2 hours","6 hours","24 hours","72 hours"]'),
 (1, 'CP-08', '{"bps":200,"min_poisha":5000}'), (1, 'CP-09', '{"window":"2 minutes","extension":"2 minutes"}'
    ),
 (1, 'CP-10', '"24 hours"'), (1, 'CP-11', '"24 hours"'), (1, 'CP-12', '{"strikes":2,"window":"90 days","
    suspension":"30 days"}'),
 (1, 'CP-13', '"15 minutes"'), (1, 'CP-14', '2'), (1, 'CP-15', '"7 days"'), (1, 'CP-16', '"3 days"'), (1, 'CP
    -17', '"7 days"'),
 (1, 'CP-18', '"3 days"'), (1, 'CP-19', '"5 days"'), (1, 'CP-20', '"48 hours"'), (1, 'CP-21', '"5 days"'),
 (1, 'CP-22', '"24 hours"'), (1, 'CP-23', '"24 hours"'), (1, 'CP-24', '"7 days"'),
 (1, 'CP-25', '{"standard_bps":500,"pro_bps":300}'), (1, 'CP-26', '1000'),
 (1, 'CP-27', '{"new_days":30,"new_per_order":5000000,"new_open":10000000,"per_order":20000000}'),
 (1, 'CP-28', '{"standard":10,"pro":50}'), (1, 'CP-29', '"24 hours"'),
 (1, 'CP-30', '["kyc.revoke","blacklist.add","blacklist.remove","dispute.outcome_20000","account.reinstate","
    staff.mfa_reset","emergency.control"]'),
 (1, 'CP-31', '"5 minutes"'), (1, 'CP-32', '{"tolerance":"5 minutes","retention":"30 days"}'),
 (1, 'CP-33', '{"image_bytes":10485760,"photos":10,"evidence":10}'), (1, 'CP-34', '3'),
 (1, 'CP-35', '{"bids_per_min":10,"chat_per_min":30,"listings_per_day":10,"reports_per_day":20,"
    search_per_min":60}'),
 (1, 'CP-36', 'true'), (1, 'CP-37', '{"periods":["24 hours","48 hours"],"fee_poisha":null}'),
 (1, 'CP-38', '{"auction_end":"5 minutes","reminders":["24 hours","2 hours"]}'),
 (1, 'CP-39', '{"window":"14 days","edit":"48 hours"}'), (1, 'CP-40', '"1 year"'), (1, 'CP-41', '"30 days"'),
 (1, 'CP-42', '{"critical":"7 days","high":"30 days"}'), (1, 'CP-43', '"see delivery.fare_tables"'), (1, 'CP
    -44', '"48 hours"'),
 (1, 'CP-45', '5'), (1, 'CP-46', '"15 minutes"'), (1, 'CP-47', '{"strikes":3,"window":"90 days"}'),
 (1, 'SW-new_orders', 'true'), (1, 'SW-payouts', 'true');

-- ====================================================================
-- Grants (SEC-INF-03)
-- ====================================================================
GRANT USAGE ON SCHEMA disputes, trust, staff TO tb_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA disputes, trust, staff TO tb_app;
REVOKE UPDATE ON disputes.history, disputes.statements, disputes.evidence, trust.strikes,
                 staff.config_versions, staff.config_values FROM tb_app;
GRANT SELECT ON trust.profile_summary TO tb_app;
```